

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

ĐẠI HỌC BÁCH KHOA HÀ NỘI

ĐỒ ÁN TỐT NGHIỆP

**Nghiên cứu và ứng dụng giao thức điều phối phi tập
trung cho MLS trên mạng ngang hàng**

LÊ TIẾN DŨNG

dung.lt224837@sis.hust.edu.vn

Chương trình đào tạo: Khoa học máy tính

Giảng viên hướng dẫn: TS. Đỗ Bá Lâm _____

Chữ kí GVHD

Khoa: Khoa học máy tính

Trường: Công nghệ Thông tin và Truyền thông

HÀ NỘI, 06/2026

LỜI CẢM ƠN

Trước hết, em xin bày tỏ lòng biết ơn chân thành nhất tới TS. Đỗ Bá Lâm. Trong suốt quá trình thực hiện đề tài, thầy đã luôn dành thời gian định hướng khoa học, tận tình hướng dẫn và hỗ trợ em vượt qua những khó khăn từ giai đoạn bắt đầu cho đến khi hoàn thành đồ án tốt nghiệp này.

Đồng thời, em xin trân trọng cảm ơn tập thể giảng viên Đại học Bách khoa Hà Nội nói chung và các thầy cô giáo tại Trường Công nghệ Thông tin và Truyền thông nói riêng. Những kiến thức chuyên môn sâu sắc cùng phương pháp luận nghiên cứu khoa học mà các thầy cô truyền đạt trong suốt quá trình học tập là hành trang vô cùng quý giá cho em.

Cuối cùng, em xin gửi lời cảm ơn đến gia đình và bạn bè. Đây là nguồn động viên tinh thần to lớn, luôn chia sẻ, đồng hành và tạo mọi điều kiện thuận lợi nhất để em yên tâm tập trung học tập và hoàn thành chặng đường nghiên cứu này.

TÓM TẮT NỘI DUNG ĐỒ ÁN

Trong bối cảnh rủi ro an ninh mạng ngày càng phức tạp, việc bảo vệ các hệ thống liên lạc nội bộ đang trở thành ưu tiên hàng đầu, thúc đẩy sự phổ biến của tiêu chuẩn mã hóa đầu cuối (E2EE) như một giải pháp bảo mật tất yếu. Hiện nay, hầu hết các ứng dụng nhắn tin E2EE hàng đầu vận hành dựa trên giao thức Double Ratchet, một tiêu chuẩn bộc lộ nhiều hạn chế khi mở rộng quy mô nhóm. Nhằm giải quyết bài toán này, Messaging Layer Security (MLS) ra đời như một giao thức tiên phong, tái định nghĩa toàn diện kiến trúc mã hóa nhóm. Tuy nhiên, tiêu chuẩn MLS hiện tại được thiết kế độc quyền cho kiến trúc tập trung và bắt buộc phụ thuộc vào một máy chủ điều phối để đảm bảo thứ tự. Việc triển khai MLS trên môi trường mạng ngang hàng (P2P) phi tập trung vẫn là một thách thức chưa được giải quyết.

Từ thực tiễn đó, em quyết định theo đuổi hướng nghiên cứu tích hợp trực tiếp giao thức MLS lên kiến trúc mạng ngang hàng, tước bỏ hoàn toàn máy chủ trung tâm. Hướng đi này được lựa chọn nhằm chứng minh khả năng dung hợp giữa tiêu chuẩn mã hóa nhóm tiên tiến nhất hiện nay và triết lý mạng phi tập trung, từ đó vạch ra mô hình lý thuyết cho một hệ thống truyền thông không tồn tại điểm lỗi duy nhất.

Tổng quan giải pháp của nghiên cứu là đề xuất một lớp điều phối phi tập trung. Kiến trúc này thay thế hoàn toàn vai trò của máy chủ trong việc quản lý cập nhật trạng thái nhóm, đồng thời cung cấp khả năng tự động duy trì trạng thái mã hóa nhóm, nhận diện và hàn gắn phân nhánh khi mạng xảy ra đứt gãy. Đóng góp cốt lõi của đồ án là thiết lập thành công giải pháp hoàn chỉnh cho mô hình ứng dụng MLS trong mạng ngang hàng, sau cùng là một ứng dụng thử nghiệm vận hành ổn định, trực tiếp xác thực tính đúng đắn của nghiên cứu và mở ra nền tảng tham chiếu vững chắc cho các hệ thống bảo mật tương lai.

Sinh viên thực hiện

Lê Tiến Dũng

MỤC LỤC

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI.....	1
1.1 Đặt vấn đề.....	1
1.2 Các giải pháp hiện tại và hạn chế	2
1.3 Mục tiêu và định hướng giải pháp	3
1.4 Đóng góp của đề án	4
1.5 Bố cục đề án	4
CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT	5
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán	5
2.2 Các kết quả nghiên cứu tương tự	7
2.2.1 Các cơ chế mã hóa nhóm trước MLS.....	7
2.2.2 MLS với Delivery Service	8
2.2.3 Các hướng phi tập trung hóa MLS	8
2.3 Nền tảng MLS và TreeKEM	9
2.3.1 Các khái niệm cốt lõi: client, group, member và epoch	9
2.3.2 Ratchet Tree và TreeKEM.....	9
2.3.3 Proposal, Commit và tiến trình chuyển epoch	9
2.3.4 Các dấu hiệu nhận biết trạng thái: tree hash, transcript hash và epoch authenticator	10
2.3.5 Application messages, delayed messages và External Commit.....	10
2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán	11
2.4.1 Đặc tính của mạng ngang hàng.....	11
2.4.2 go-libp2p và GossipSub.....	11
2.4.3 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng	12
2.4.4 Khoảng trống cần giải quyết.....	12

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT.....	14
3.1 Tổng quan giải pháp.....	14
3.1.1 Bài toán cần giải quyết	14
3.1.2 Kiến trúc tổng thể	15
3.1.3 Các thành phần chính của giao thức	16
3.1.4 Luồng hoạt động tổng thể	16
3.2 Giao thức người ghi duy nhất theo Epoch.....	17
3.2.1 Mục tiêu thiết kế	17
3.2.2 Tập ứng viên hợp lệ	17
3.2.3 Hàm bầu chọn Token Holder.....	18
3.2.4 Tạo Proposal và Commit.....	19
3.2.5 Chuyển quyền và failover	20
3.3 Kiểm tra nhất quán Epoch	21
3.3.1 Mục tiêu thiết kế	21
3.3.2 Lớp bao điều phối (Coordination Envelope).....	21
3.3.3 Phân loại thông điệp theo Epoch.....	22
3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai	24
3.3.5 Xử lý Proposal bị bỏ sót.....	24
3.4 Cơ chế phục hồi phân nhánh trạng thái	25
3.4.1 Bối cảnh network partition.....	25
3.4.2 Phát hiện fork bằng heartbeat trạng thái	25
3.4.3 Hàm trọng số nhánh	26
3.4.4 Quy trình Fork Healing.....	28
3.4.5 Lưu trạng thái bền vững trong quá trình hàn gắn	29
3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock	30
3.5.1 Lý do cần HLC	30

3.5.2 Tạo timestamp khi gửi tin.....	30
3.5.3 Cập nhật HLC khi nhận tin	31
3.5.4 Thứ tự hiển thị tất định	32
3.6 Quản lý trạng thái Go-Rust và lưu trữ cục bộ.....	33
3.6.1 Lý do sử dụng Rust Sidecar phi trạng thái.....	33
3.6.2 Phân chia trách nhiệm giữa SQLite và Rust Sidecar.....	33
3.6.3 Luồng xử lý khi gửi tin nhắn ứng dụng.....	33
3.6.4 Luồng xử lý khi thay đổi thành viên.....	34
3.7 Tính đúng đắn trực giác của phương pháp	34
3.8 Kết chương.....	35
CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT	36
4.1 Mục tiêu và phạm vi phân tích	36
4.2 Mô hình phân tích.....	36
4.3 Bất biến 1: Epoch cục bộ không được thoái lui	37
4.3.1 Phát biểu bất biến.....	37
4.3.2 Cơ sở từ MLS	37
4.3.3 Cơ chế bảo toàn bất biến.....	38
4.4 Bất biến 2: Với cùng một view, chỉ có một Token Holder.....	39
4.4.1 Phát biểu bất biến.....	39
4.4.2 Ý nghĩa đối với concurrent commits	40
4.5 Bất biến 3: Fork trạng thái phải phát hiện được.....	40
4.5.1 Hiện tượng fork trạng thái mật mã	40
4.5.2 Fingerprint trạng thái.....	40
4.6 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh....	41
4.6.1 Vấn đề của việc gộp trạng thái	41
4.6.2 Quy tắc chọn nhánh.....	42

4.6.3 Giới hạn của lập luận.....	43
4.7 Xử lý thông điệp phát sinh trong nhánh thua.....	43
4.8 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit.....	44
4.8.1 Phân biệt crypto ordering và application ordering	44
4.8.2 Cơ sở của HLC	44
4.8.3 Giới hạn của HLC trong đồ án.....	45
4.9 Phân tích chi phí lý thuyết	45
4.9.1 Chi phí gửi application message	45
4.9.2 Chi phí cập nhật thành viên và rekey	46
4.9.3 Chi phí của Single-Writer	46
4.9.4 Chi phí của Epoch Checks và Fork Detection.....	46
4.9.5 Chi phí của Fork Healing	47
4.9.6 Ảnh hưởng của kiến trúc sidecar phi trạng thái	47
4.10 Tổng hợp các bất biến và cơ chế bảo toàn.....	47
4.11 Giới hạn lý thuyết của phương pháp.....	48
4.12 Kết chương.....	49
CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM.....	50
5.1 Mục tiêu và phạm vi đánh giá	50
5.2 Môi trường và phương pháp thí nghiệm	51
5.3 Đánh giá hội tụ trong điều kiện mạng hỗn loạn.....	52
5.4 Đánh giá khả năng phục hồi sau phân vùng mạng.....	52
5.5 Đánh giá cơ chế Single-Writer và gom Proposal	54
5.6 Đánh giá khả năng mở rộng của thao tác thành viên.....	55
5.7 Đánh giá chi phí mật mã MLS	55
5.8 Chi phí phụ trợ của lớp điều phối	56

5.9 Kiểm chứng Forward Secrecy bằng Rust sidecar thật	57
5.10 Nhận xét và giới hạn đánh giá	57
CHƯƠNG 6. KẾT LUẬN	59
6.1 Kết luận	59
6.2 Hướng phát triển trong tương lai	60
TÀI LIỆU THAM KHẢO.....	63

DANH MỤC HÌNH VẼ

Hình 2.1	Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS . . .	6
Hình 3.1	Kiến trúc triển khai chi tiết của Go Host và Rust Sidecar . . .	15
Hình 5.1	Hội tụ epoch của các nút trong chaos test	52
Hình 5.2	Thời gian phục hồi khi độ dài phân vùng thay đổi	53
Hình 5.3	Hiệu quả của cơ chế gom Proposal khi mức đồng thời tăng . .	54
Hình 5.4	Phân rã chi phí phụ trợ của lớp điều phối	56

DANH MỤC BẢNG BIỂU

Bảng 4.1	Quy tắc xử lý thông điệp theo epoch	38
Bảng 4.2	Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn	48
Bảng 5.1	Các tiêu chí đánh giá thực nghiệm	51
Bảng 5.2	Thời gian phục hồi sau phân vùng mạng	53
Bảng 5.3	So sánh baseline và cơ chế gom Proposal	54
Bảng 5.4	Chi phí thao tác thành viên theo quy mô nhóm	55
Bảng 5.5	So sánh độ trễ mật mã giữa các mô hình	56

DANH MỤC THUẬT NGỮ VÀ TỪ VIẾT TẮT

Thuật ngữ	Ý nghĩa
BFT	Chịu lỗi Byzantine (Byzantine Fault Tolerance)
DHT	Bảng băm phân tán (Distributed Hash Table)
DMLS	Bảo mật tầng thông điệp phi tập trung (Decentralized Messaging Layer Security)
DS	Dịch vụ phân phối (Delivery Service)
E2EE	Mã hóa đầu cuối (End-to-End Encryption)
FREEK	Thỏa thuận khóa nhóm liên tục chịu rẽ nhánh (Fork-Resilient Continuous Group Key Agreement)
FS	Bảo mật xuôi (Forward Secrecy)
HLC	Đồng hồ logic hỗn hợp (Hybrid Logical Clock)
IP	Giao thức Internet (Internet Protocol)
LAN	Mạng cục bộ (Local Area Network)
mDNS	Hệ thống phân giải tên miền multicast (multicast Domain Name System)
MLS	Bảo mật tầng thông điệp (Messaging Layer Security)
NAT	Biến đổi địa chỉ mạng (Network Address Translation)
P2P	Mạng ngang hàng (Peer-to-Peer)
PCS	Bảo mật sau thỏa hiệp (Post-Compromise Security)
QUIC	Kết nối Internet UDP nhanh (Quick UDP Internet Connections)
RFC	Tài liệu đặc tả chuẩn / Yêu cầu bình luận (Request for Comments)

Thuật ngữ	Ý nghĩa
TCP	Giao thức điều khiển truyền vận (Transmission Control Protocol)
WAN	Mạng diện rộng (Wide Area Network)

CHƯƠNG 1. GIỚI THIỆU ĐỀ TÀI

1.1 Đặt vấn đề

Trong bối cảnh hoạt động trao đổi thông tin ngày càng phụ thuộc vào hạ tầng số, yêu cầu bảo vệ nội dung liên lạc nội bộ đang trở nên cấp thiết đối với cơ quan, doanh nghiệp và các tổ chức có dữ liệu nhạy cảm. Rò rỉ thông tin không chỉ gây thiệt hại về kinh tế, mà còn ảnh hưởng trực tiếp tới uy tín và an toàn vận hành. Vì vậy, mã hóa đầu cuối đã trở thành một hướng tiếp cận quan trọng, trong đó chỉ các thiết bị đầu cuối hợp lệ mới có khả năng giải mã nội dung thông điệp, còn các thành phần trung gian không được phép truy cập bản rõ [1], [2].

Tuy nhiên, trong nhiều hệ thống nhắn tin bảo mật hiện nay, máy chủ trung tâm vẫn giữ vai trò rất lớn trong quá trình vận hành. Máy chủ thường đảm nhiệm việc định tuyến thông điệp, hỗ trợ thiết bị ngoại tuyến, lưu trữ vật liệu khóa ban đầu và đồng bộ trạng thái giữa các thành viên [2]. Cách tổ chức này giúp hệ thống dễ quản lý hơn, nhưng đồng thời cũng tạo ra sự phụ thuộc đáng kể vào hạ tầng tập trung. Với những môi trường như mạng nội bộ biệt lập, mạng tạm thời không có Internet, hoặc các tổ chức muốn kiểm soát chặt chẽ dữ liệu, sự phụ thuộc đó trở thành một hạn chế thực tế. Những bối cảnh này đòi hỏi một mô hình trong đó dữ liệu được ưu tiên lưu tại thiết bị, các nút có thể trao đổi trực tiếp với nhau, và hệ thống vẫn duy trì được hoạt động ngay cả khi không có máy chủ trung tâm luôn sẵn sàng. Định hướng đó gần với tinh thần của các hệ thống Local-First và mạng ngang hàng [3].

Đối với liên lạc hai bên, nhiều giao thức đã chứng minh được hiệu quả trong thực tế, tiêu biểu là Double Ratchet. Tuy nhiên, khi mở rộng sang liên lạc nhóm, bài toán không còn dừng ở việc mã hóa từng thông điệp, mà chuyển thành bài toán duy trì một trạng thái mật mã chung giữa nhiều thiết bị có thể gửi, nhận, mất kết nối và quay trở lại ở những thời điểm khác nhau [1], [4]. Nói cách khác, khó khăn cốt lõi của truyền thông nhóm bảo mật không chỉ nằm ở bí mật nội dung, mà còn nằm ở việc bảo đảm tất cả thành viên hợp lệ cùng nhìn thấy một trạng thái khóa nhóm nhất quán.

Messaging Layer Security (MLS) được chuẩn hóa trong RFC 9420 để giải quyết bài toán thiết lập khóa nhóm bất đồng bộ với các bảo đảm quan trọng như Forward Secrecy và Post-Compromise Security [1]. Nhờ cấu trúc TreeKEM, MLS cho phép cập nhật trạng thái khóa nhóm hiệu quả hơn nhiều so với các cơ chế mã hóa nhóm dựa hoàn toàn trên các kênh hai bên. Dù vậy, MLS không phải là một hệ thống nhắn tin hoàn chỉnh. Theo kiến trúc của RFC 9750, giao thức này vẫn cần được đặt trong một môi trường triển khai cụ thể, nơi ứng dụng phải tự giải quyết các vấn đề

như xác thực danh tính, phân phối thông điệp và đặc biệt là xử lý thứ tự các bản tin làm thay đổi trạng thái nhóm [2].

Chính ở điểm này xuất hiện mâu thuẫn trung tâm của đề tài. Trong mô hình có Delivery Service nhất quán mạnh, thành phần phân phối có thể đóng vai trò áp đặt thứ tự tuyến tính cho các Commit và loại bỏ xung đột khi nhiều thành viên cùng cập nhật nhóm tại một epoch. Ngược lại, trong môi trường P2P không có một thực thể trung tâm làm bộ tuần tự hóa, các Commit có thể đến các thiết bị theo những thứ tự khác nhau. Hệ quả là các nút có thể áp dụng những chuỗi thay đổi không giống nhau và làm nhóm rơi vào trạng thái phân nhánh mật mã, tức là các thành viên tuy vẫn mang cùng định danh nhóm nhưng không còn chia sẻ cùng trạng thái khóa để tiếp tục liên lạc [2], [5], [6].

Từ thực tế đó, đề án lựa chọn nghiên cứu bài toán vận hành MLS trên mạng ngang hàng theo hướng bổ sung một lớp điều phối phi tập trung ở tầng ứng dụng. Mục tiêu của hướng tiếp cận này không phải là sửa đổi lõi mật mã của MLS, mà là xây dựng các cơ chế hỗ trợ để nhóm vẫn có thể duy trì tính nhất quán trạng thái, phát hiện phân nhánh khi cần thiết và hội tụ trở lại sau khi mạng được khôi phục.

1.2 Các giải pháp hiện tại và hạn chế

Các hướng tiếp cận hiện nay đối với truyền thông nhóm bảo mật có thể chia thành hai nhóm lớn. Nhóm thứ nhất mở rộng từ các cơ chế bảo mật hai bên sang môi trường nhóm; nhóm thứ hai sử dụng giao thức thiết lập khóa nhóm chuyên biệt, trong đó MLS là đại diện tiêu biểu [1], [7], [8]. Mỗi hướng đều có giá trị thực tiễn nhất định, nhưng khi đặt trong yêu cầu vận hành P2P hoàn toàn phi tập trung, các giới hạn của chúng bộc lộ rõ.

Với nhóm thứ nhất, những cơ chế như Sender Keys hay Megolm có ưu điểm là triển khai tương đối thực dụng và có chi phí gửi tin thấp khi nhóm đông thành viên [7], [8]. Tuy nhiên, chúng chủ yếu tối ưu cho việc phát tán thông điệp trong nhóm, chứ không trực tiếp giải quyết bài toán duy trì một trạng thái khóa nhóm nhất quán khi thành viên thay đổi thường xuyên hoặc khi mạng bị chia cắt. Trong các trường hợp khóa gửi tin bị lộ hoặc membership biến động liên tục, chi phí rekey và khả năng phục hồi bảo mật của các cơ chế này vẫn còn những hạn chế đáng kể [7], [8].

Với nhóm thứ hai, MLS cung cấp một mô hình chặt chẽ hơn cho quản lý khóa nhóm và đạt được các bảo đảm bảo mật mạnh hơn cho truyền thông quy mô lớn [1]. Dù vậy, MLS chuẩn vẫn giả định hệ thống triển khai có khả năng bảo đảm việc phân phối và tuần tự hóa các bản tin thay đổi trạng thái [2]. Khi đưa MLS sang môi trường P2P, bài toán khó nhất không nằm ở bản thân thao tác mật mã, mà nằm ở việc thay thế vai trò điều phối vốn thường do Delivery Service đảm nhiệm.

Một số nghiên cứu gần đây, chẳng hạn hướng Decentralized MLS, đã thử giải quyết vấn đề bằng cách mở rộng trực tiếp giao thức MLS để hỗ trợ xử lý Commit ngoài thứ tự trong môi trường phi tập trung [6]. Đây là một hướng nghiên cứu đáng chú ý, nhưng cũng đòi hỏi client duy trì thêm trạng thái mật mã cũ hoặc trạng thái tạm thời, từ đó làm tăng độ phức tạp triển khai và đặt ra thêm đánh đổi đối với Forward Secrecy [5], [6]. Hơn nữa, hướng này hiện vẫn dừng ở mức Internet-Draft, chưa trở thành một chuẩn hoàn chỉnh.

Từ các phân tích trên có thể thấy rằng khoảng trống của bài toán không nằm ở việc thiếu một giao thức mã hóa nhóm, mà nằm ở việc thiếu một cơ chế điều phối đủ nhẹ, đủ thực dụng và phù hợp với đặc tính của mạng ngang hàng. Đây cũng chính là khoảng trống mà đề án hướng tới giải quyết.

1.3 Mục tiêu và định hướng giải pháp

Mục tiêu của đề án là nghiên cứu, thiết kế và hiện thực một cơ chế điều phối phi tập trung nhằm hỗ trợ MLS vận hành trên mạng ngang hàng. Về mặt nghiên cứu, đề án tập trung vào ba vấn đề chính: giảm khả năng xuất hiện concurrent commits khi các nút còn cùng góc nhìn về nhóm, bảo vệ tính nhất quán epoch của trạng thái cục bộ trong môi trường bất đồng bộ, và xây dựng cơ chế phục hồi khi phân nhánh trạng thái vẫn xảy ra do network partition. Về mặt ứng dụng, đề án hướng tới một nguyên mẫu liên lạc nhóm bảo mật hoạt động theo định hướng Local-First, có khả năng vận hành trong các điều kiện mạng không ổn định và giảm phụ thuộc vào hạ tầng tập trung [3].

Trên cơ sở mục tiêu đó, đề án lựa chọn cách tiếp cận giữ nguyên lõi mật mã của MLS/OpenMLS, đồng thời bổ sung một lớp điều phối ở phía ngoài để xử lý các vấn đề mà môi trường P2P đặt ra. Thay vì mở rộng trực tiếp giao thức MLS hoặc đưa vào một cơ chế đồng thuận nặng cho mọi Commit, giải pháp đề xuất tập trung vào bốn nội dung: giới hạn quyền tạo Commit theo từng epoch, kiểm tra tính phù hợp của thông điệp trước khi đưa xuống lớp mật mã, phát hiện và hàn gắn phân nhánh trạng thái khi mạng bị chia cắt, và sử dụng đồng hồ logic lai để ổn định thứ tự hiển thị thông điệp ứng dụng. Cách tiếp cận này cho phép hệ thống tận dụng các bảo đảm bảo mật của MLS, đồng thời thích nghi tốt hơn với đặc tính bất đồng bộ và phân tán của mạng ngang hàng.

Trong lựa chọn thiết kế này, đề án chấp nhận một đánh đổi có chủ đích. Hệ thống không theo đuổi nhất quán mạnh tuyệt đối tại mọi thời điểm, bởi điều đó thường đòi hỏi chi phí đồng thuận cao và làm giảm tính sẵn sàng của các phân vùng mạng không đủ điều kiện liên lạc [9]. Thay vào đó, đề án ưu tiên khả năng hoạt động cục bộ trong điều kiện mạng chia cắt, nhưng vẫn đặt ra cơ chế để các nút có thể hội tụ

trở lại một trạng thái MLS hợp lệ sau khi kết nối được khôi phục. Đây là hướng đi phù hợp hơn với mục tiêu P2P và Local-First của hệ thống.

1.4 Đóng góp của đề án

Trên cơ sở định hướng nêu trên, đề án có bốn đóng góp chính. Thứ nhất, đề án xác định bài toán trung tâm của MLS trên mạng ngang hàng dưới góc nhìn nhất quán trạng thái mật mã, qua đó chỉ ra rằng khó khăn cốt lõi không nằm ở việc truyền thông điệp, mà nằm ở việc duy trì một chuỗi trạng thái nhóm hợp lệ khi không còn Delivery Service trung tâm. Thứ hai, đề án đề xuất một lớp điều phối phi tập trung bao quanh MLS, trong đó các cơ chế Single-Writer, kiểm tra epoch, phát hiện phân nhánh và phục hồi sau phân mảnh được phối hợp để giảm nguy cơ fork và hỗ trợ hệ thống hội tụ trở lại. Thứ ba, đề án xây dựng một nguyên mẫu phần mềm kết hợp mạng P2P, lưu trữ cục bộ và lõi mật mã MLS/OpenMLS, qua đó kiểm tra khả năng triển khai của hướng tiếp cận đã đề xuất. Thứ tư, đề án cung cấp các kết quả đánh giá ban đầu về tính hội tụ, khả năng phục hồi sau network partition và tác động của lớp điều phối đối với quá trình vận hành MLS trong môi trường mạng ngang hàng.

1.5 Bố cục đề án

Phần còn lại của báo cáo được tổ chức như sau. Chương 2 trình bày cơ sở lý thuyết của đề tài, bao gồm nền tảng MLS, TreeKEM, vai trò của Delivery Service, các hướng tiếp cận liên quan và những đặc tính của môi trường mạng ngang hàng có ảnh hưởng trực tiếp tới bài toán nhất quán trạng thái. Trên nền tảng đó, Chương 3 trình bày chi tiết giao thức điều phối phi tập trung được đề xuất, làm rõ các thành phần, nguyên tắc hoạt động và cách phối hợp giữa lớp điều phối với lõi mật mã MLS.

Tiếp theo, Chương 4 phân tích cơ sở lý thuyết của giải pháp theo hướng các bất biến quan trọng, điều kiện hội tụ và giới hạn của phương pháp trong môi trường phân tán bất đồng bộ. Chương 5 trình bày quá trình đánh giá thực nghiệm trên nguyên mẫu triển khai, tập trung vào khả năng hội tụ, phục hồi sau phân mảnh mạng và chi phí vận hành của hệ thống. Cuối cùng, Chương 6 tổng kết các kết quả đạt được, chỉ ra những hạn chế còn tồn tại và đề xuất các hướng phát triển tiếp theo của đề án.

CHƯƠNG 2. NỀN TẢNG LÝ THUYẾT

Chương 1 đã trình bày bối cảnh, mục tiêu và định hướng chung của đề tài. Tiếp nối phần mở đầu đó, Chương 2 tập trung làm rõ nền tảng lý thuyết cần thiết cho bài toán vận hành MLS trên mạng ngang hàng. Trọng tâm của chương không phải là liệt kê thật nhiều kiến thức rời rạc, mà là chỉ ra mối liên hệ giữa ba lớp vấn đề: lời mật mã của MLS, đặc tính bất đồng bộ của mạng P2P và yêu cầu duy trì một trạng thái nhóm nhất quán.

Nội dung chương được tổ chức thành bốn phần. Trước hết, chương phân tích ngữ cảnh bài toán để làm rõ vì sao khi đưa MLS từ mô hình có máy chủ trung tâm sang mô hình ngang hàng, khó khăn lớn nhất không nằm ở việc truyền tin mà nằm ở việc giữ cho các thành viên cùng đi trên một chuỗi trạng thái mật mã hợp lệ. Tiếp theo, chương khảo sát các hướng nghiên cứu liên quan nhằm chỉ ra ưu điểm, giới hạn và khoảng trống còn lại. Sau đó, chương trình bày các kiến thức nền tảng quan trọng nhất của MLS và TreeKEM. Cuối cùng, chương giới thiệu ngắn gọn những đặc tính của mạng ngang hàng, cơ chế phát tán thông điệp và công cụ hỗ trợ sắp xếp thông điệp ứng dụng trong hệ phân tán.

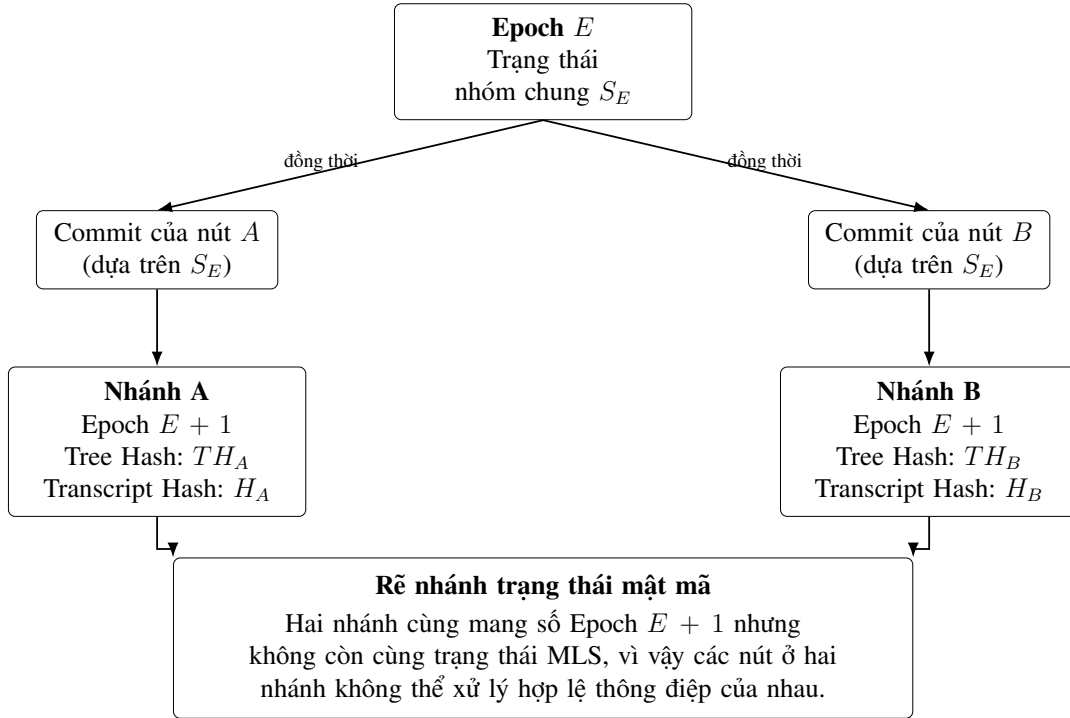
2.1 Ngữ cảnh của bài toán: nhất quán trạng thái mật mã trong hệ phân tán

Trong hệ phân tán nói chung, bài toán nhất quán trạng thái nhằm bảo đảm nhiều nút cùng hiểu và cùng cập nhật dữ liệu theo một quy tắc thống nhất. Đối với hệ thống nhắn tin nhóm được mã hóa đầu cuối, trạng thái cần được giữ nhất quán không chỉ là dữ liệu ứng dụng, mà còn là trạng thái mật mã dùng để sinh khóa, xác thực thành viên và ràng buộc lịch sử hội thoại [1], [2]. Vì vậy, nếu các nút không còn cùng nhìn thấy một trạng thái mật mã chung, khả năng giao tiếp của nhóm sẽ bị ảnh hưởng trực tiếp.

Trong MLS, trạng thái nhóm tiến hóa theo các kỷ nguyên, gọi là epoch. Có thể hiểu mỗi epoch là một phiên bản cụ thể của nhóm, trong đó tập thành viên hợp lệ cùng chia sẻ một bộ bí mật tương ứng. Khi nhóm thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa, trạng thái này phải chuyển sang epoch mới. Nếu tất cả thành viên cùng áp dụng các thay đổi theo cùng một thứ tự, nhóm tiếp tục hoạt động bình thường. Ngược lại, nếu các thay đổi được áp dụng theo những thứ tự khác nhau, các thành viên có thể dần tách khỏi nhau về mặt mật mã [1], [5].

Hiện tượng đó được gọi là rẽ nhánh trạng thái mật mã (Cryptographic State Fork). Đây là tình huống trong đó các nút vẫn nghĩ rằng mình thuộc cùng một nhóm, thậm chí có thể cùng nhìn thấy cùng số epoch, nhưng thực tế lại đang nắm

giữ các bí mật nhóm, hàm băm cây (tree hash) hoặc lịch sử bắt tay (transcript hash) khác nhau. Khi đó, thông điệp được tạo trên một nhánh không còn được xử lý hợp lệ trên nhánh còn lại. Khác với xung đột dữ liệu thông thường, trạng thái mật mã đã rõ nhánh không thể được ghép lại bằng cách trộn hai phiên bản dữ liệu, vì mỗi nhánh đã sinh ra một chuỗi khóa và một lịch sử xác thực riêng [1], [5], [6]. Hiện tượng này được minh họa trong Hình 2.1.



Hình 2.1: Minh họa hiện tượng rẽ nhánh trạng thái mật mã trong MLS

Trong các triển khai thông thường của MLS, việc hạn chế rẽ nhánh được hỗ trợ bởi Delivery Service (DS), có thể hiểu là lớp dịch vụ tiếp nhận và phân phối các thông điệp MLS giữa các thiết bị [2]. Trong môi trường có DS nhất quán mạnh, khi nhiều thành viên cùng tạo Commit ở cùng một epoch, hệ thống có một điểm tự nhiên để quyết định Commit nào được xử lý trước. Nhờ đó, các thay đổi trạng thái dễ được quan sát theo một thứ tự thống nhất hơn.

Khó khăn xuất hiện khi đưa MLS sang mạng ngang hàng. Trong môi trường P2P, thông điệp có thể đến chậm, đến sai thứ tự, bị lặp lại hoặc tạm thời không đến được do thiết bị ngoại tuyến hay do mạng bị chia cắt. Khi đó, không còn một thực thể trung tâm nào tự nhiên đứng ra sắp xếp thứ tự cho các Commit. Nói cách khác, MLS đòi hỏi chuỗi epoch tiến hóa gần như tuyến tính, trong khi mạng ngang hàng bất đồng bộ lại không tự cung cấp thứ tự tuyến tính toàn cục cho các thông điệp thay đổi trạng thái [2], [9]. Đây chính là mâu thuẫn cốt lõi của bài toán mà đề án cần giải quyết.

Trong phạm vi đồ án, hệ thống được xem như một mạng gồm nhiều thiết bị ngang hàng, mỗi thiết bị lưu cục bộ trạng thái MLS của các nhóm mà nó tham gia. Các tình huống được quan tâm gồm độ trễ mạng không đồng đều, thiết bị ngoại tuyến, sự biến động nút mạng và phân mảnh mạng. Mục tiêu của phần nghiên cứu không phải thay đổi các bảo đảm mật mã nền tảng của MLS, mà là tìm cách tổ chức việc điều phối ở tầng ứng dụng để các thiết bị vẫn có thể hội tụ trở lại một trạng thái nhóm hợp lệ sau những bất ổn của mạng.

2.2 Các kết quả nghiên cứu tương tự

Các nghiên cứu liên quan có thể được nhìn từ ba hướng chính: các cơ chế nhắn tin nhóm xuất hiện trước MLS, MLS trong mô hình có Delivery Service và các nỗ lực đưa MLS sang môi trường phi tập trung. Việc phân chia theo hướng này giúp làm rõ vì sao đồ án không chọn lặp lại hoàn toàn một giải pháp có sẵn, mà cần một hướng tiếp cận riêng phù hợp hơn với mục tiêu P2P và Local-First.

2.2.1 Các cơ chế mã hóa nhóm trước MLS

Trước khi MLS được chuẩn hóa, nhiều hệ thống nhắn tin nhóm đã mở rộng từ cơ chế bảo mật hai bên sang môi trường nhiều người tham gia. Một nền tảng quan trọng là Double Ratchet, vốn rất hiệu quả cho liên lạc hai bên [4]. Khi đưa sang nhóm, một hướng phổ biến là để mỗi người gửi tự tạo một khóa gửi tin riêng, rồi phân phối khóa đó cho các thành viên còn lại qua các kênh bảo mật hai bên. Cách làm này thường được gọi là Sender Keys [7]. Sau khi khóa gửi tin đã được chia sẻ, người gửi chỉ cần mã hóa một lần cho mỗi thông điệp, còn hạ tầng bên dưới chịu trách nhiệm phát tán bản mã tới cả nhóm. Ưu điểm của hướng này là chi phí gửi tin thấp và triển khai tương đối thực dụng. Tuy nhiên, khi nhóm đông hoặc thay đổi thành viên thường xuyên, chi phí cập nhật khóa và phục hồi sau thỏa hiệp vẫn tăng đáng kể.

Megolm trong hệ sinh thái Matrix cũng theo tinh thần tối ưu việc gửi tin nhóm [8]. Có thể hiểu đơn giản rằng mỗi người gửi duy trì một phiên gửi ra (*outbound session*), tức một trạng thái khóa được dùng liên tiếp để mã hóa nhiều tin nhắn trong cùng một phòng trò chuyện. Bên trong phiên đó, trạng thái khóa được cập nhật dần theo một cơ chế nhất định của nhóm (*group ratchet*). Thiết kế này giúp việc gửi tin trong nhóm nhẹ hơn, nhưng điểm đổi lại là khả năng phục hồi sau thỏa hiệp không mạnh bằng những mô hình thay khóa nhóm chặt chẽ hơn. Nếu trạng thái của một phiên bị lộ, phạm vi thông điệp bị ảnh hưởng có thể kéo dài cho đến khi hệ thống tạo phiên mới và phân phối lại khóa.

Nhìn chung, Sender Keys và Megolm đều có giá trị thực tiễn cao đối với bài toán phát tán tin nhắn trong nhóm. Tuy nhiên, chúng không trực tiếp giải quyết bài

toán mà đồ án quan tâm nhất, đó là duy trì một trạng thái nhóm nhất quán khi nhiều thiết bị có thể cùng thay đổi trạng thái trong môi trường P2P bất đồng bộ. Đây là lý do MLS trở thành nền tảng phù hợp hơn cho đề tài.

2.2.2 MLS với Delivery Service

MLS được chuẩn hóa để cung cấp cơ chế thiết lập khóa nhóm bất đồng bộ với các bảo đảm bảo mật mạnh hơn cho truyền thông nhóm quy mô lớn [1]. Điểm quan trọng của MLS là nó không chỉ tối ưu việc gửi thông điệp, mà còn mô hình hóa rõ quá trình thay đổi thành viên và cập nhật khóa của cả nhóm thông qua các khái niệm như proposal, commit và epoch.

Trong kiến trúc chuẩn của MLS, Delivery Service thường đảm nhiệm việc lưu trữ một số vật liệu khóa công khai ban đầu, phân phối thông điệp MLS và hỗ trợ quá trình truyền thông giữa các client [2]. Nhờ có một đầu mối như vậy, hệ thống triển khai theo mô hình Client-Server có điều kiện thuận lợi hơn để quan sát các Commit theo một thứ tự đủ nhất quán. Vì thế, MLS hoạt động tự nhiên hơn trong môi trường có DS, nhưng khi bỏ đi điểm điều phối này thì bài toán ordering của Commit lập tức trở nên khó hơn nhiều.

2.2.3 Các hướng phi tập trung hóa MLS

Một hướng tiếp cận trực tiếp là bổ sung một lớp đồng thuận phân tán để thống nhất thứ tự Commit. Các cơ chế như BFT, blockchain hoặc đồng thuận dựa trên quorum có thể tạo ra một nhật ký tuyến tính chung cho các thay đổi trạng thái nhóm. Nếu mọi Commit đều được ghi vào cùng một chuỗi thứ tự, nguy cơ rẽ nhánh trạng thái sẽ giảm rõ rệt. Tuy nhiên, đổi lại là chi phí truyền thông cao, phụ thuộc vào quorum và độ phức tạp triển khai lớn. Với mục tiêu xây dựng một hệ thống P2P nhẹ, có thể vận hành trong mạng nội bộ hoặc mạng không ổn định, đây không phải lựa chọn phù hợp [3], [9].

Một hướng khác là Decentralized MLS (DMLS) của Kohbrok, trong đó chính bản thân MLS được mở rộng để xử lý Commit đến ngoài thứ tự [6]. Về ý tưởng, client giữ lại thêm một phần vật liệu khóa hoặc trạng thái tạm thời để có thể xử lý những nhánh Commit khác nhau khi mạng phi tập trung không cung cấp ordering rõ ràng. Cách làm này cho thấy một hướng nghiên cứu đáng chú ý, nhưng đồng thời cũng làm tăng độ phức tạp triển khai và đặt ra thêm đánh đổi đối với Forward Secrecy [5], [6]. Hơn nữa, hướng này hiện vẫn ở mức Internet-Draft, chưa trở thành chuẩn hoàn chỉnh.

Từ các hướng trên có thể thấy khoảng trống còn lại nằm ở chỗ sau: cần một cơ chế đủ nhẹ để phù hợp với P2P, nhưng vẫn đủ chặt để giảm nguy cơ rẽ nhánh và hỗ trợ hệ thống hội tụ trở lại sau khi mạng bị chia cắt. Đó cũng là động lực trực tiếp

cho hướng tiếp cận mà đề án theo đuổi ở các chương sau.

2.3 Nền tảng MLS và TreeKEM

2.3.1 Các khái niệm cốt lõi: client, group, member và epoch

Trong MLS, client là thực thể nắm giữ khóa mật mã và tham gia vào tiến trình thiết lập trạng thái nhóm. Group là một nhóm logic gồm các client cùng chia sẻ một trạng thái mật mã tại một thời điểm. Member là client đang thực sự nằm trong trạng thái chia sẻ đó và có quyền truy cập các bí mật của nhóm. Epoch, như đã trình bày ở trên, là một phiên bản cụ thể của trạng thái nhóm [1].

Điểm quan trọng cần nhấn mạnh là trạng thái nhóm trong MLS không phải một tập dữ liệu rời rạc có thể ghép lại tùy ý. Nó là một chuỗi epoch nối tiếp nhau, trong đó mỗi epoch mới được sinh ra từ epoch trước đó. Vì vậy, nếu các thành viên không cùng nhìn nhận một chuỗi epoch tương thích, nhóm sẽ mất khả năng duy trì một trạng thái mật mã chung.

2.3.2 Ratchet Tree và TreeKEM

TreeKEM là cơ chế trung tâm giúp MLS cập nhật khóa nhóm hiệu quả. Thay vì để mỗi thành viên phải trao đổi khóa riêng với tất cả các thành viên còn lại, MLS tổ chức các thành viên thành các lá của một cây nhị phân, gọi là Ratchet Tree. Có thể hiểu trực quan rằng mỗi lần nhóm thay đổi, hệ thống chỉ cần cập nhật một số bí mật dọc theo một đường trong cây, thay vì phát lại toàn bộ khóa cho cả nhóm [1].

Khi một thành viên cập nhật khóa, thành viên đó tạo ra một đường dẫn cập nhật (*update path*) từ lá của mình lên gốc cây. Các bí mật mới trên đường đi này được mã hóa cho các nhánh đồng cấp tương ứng, thường được gọi là *copath*. Nhờ cấu trúc cây, số lượng phép tính và lượng dữ liệu cập nhật chỉ tăng gần theo logarit của số thành viên trong nhóm. Nói ngắn gọn, TreeKEM là cơ chế giúp MLS thay khóa nhóm theo cách có cấu trúc và tiết kiệm hơn so với việc phân phối lại khóa theo kiểu tuyến tính cho toàn bộ thành viên.

Cần lưu ý rằng bí mật ở gốc cây không được dùng trực tiếp để mã hóa thông điệp ứng dụng. Sau mỗi Commit, MLS còn đưa các bí mật mới qua lịch trình sinh khóa (*key schedule*) để tạo ra các khóa cụ thể cho từng mục đích như xác thực, mã hóa thông điệp hay xuất vật liệu khóa [1]. Vì vậy, TreeKEM nên được hiểu là cơ chế cập nhật trạng thái khóa nhóm, còn việc sinh các khóa sử dụng trực tiếp cho thông điệp được xử lý ở bước sau.

2.3.3 Proposal, Commit và tiến trình chuyển epoch

MLS phân biệt rõ proposal và commit. Proposal là thông điệp đề xuất một thay đổi cho nhóm, chẳng hạn thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa.

Bản thân proposal chưa làm thay đổi epoch. Commit mới là thông điệp thực sự áp dụng một tập proposal và đưa nhóm sang epoch mới [1].

Nhìn dưới góc độ hệ phân tán, có thể xem Commit như thao tác ghi vào trạng thái mật mã của nhóm. Mỗi Commit đều làm xuất hiện một phiên bản trạng thái mới. Vì thế, nếu tại cùng một epoch mà có nhiều Commit cạnh tranh, nguy cơ rẽ nhánh trạng thái sẽ xuất hiện. Đây là lý do bài toán ordering của Commit luôn giữ vai trò trung tâm khi vận hành MLS trên mạng ngang hàng.

2.3.4 Các dấu hiệu nhận biết trạng thái: tree hash, transcript hash và epoch authenticator

Để kiểm tra các thành viên có còn cùng một trạng thái hay không, MLS sử dụng một số giá trị ràng buộc trạng thái [1]. Trong đó, hàm băm cây (tree hash) phản ánh cấu trúc Ratchet Tree hiện tại; hàm băm lịch sử bắt tay (transcript hash) ràng buộc chuỗi thông điệp bắt tay đã được xác nhận; còn bộ xác thực epoch (epoch authenticator) có thể được dùng để đối chiếu việc các thành viên có thực sự đang chia sẻ cùng trạng thái epoch hay không.

Ý nghĩa của các giá trị này đối với đồ án là rất trực tiếp. Trong môi trường P2P, hai thiết bị có thể cùng báo cùng mã nhóm và cùng số epoch, nhưng điều đó chưa đủ để kết luận rằng chúng còn ở cùng trạng thái MLS. Nếu tree hash hoặc transcript hash khác nhau, về bản chất chúng đã tách thành hai nhánh. Vì vậy, khi nghiên cứu bài toán phát hiện phân nhánh và đồng bộ lại trạng thái, các dấu hiệu nhận biết này đóng vai trò đặc biệt quan trọng.

2.3.5 Application messages, delayed messages và External Commit

MLS phân biệt thông điệp bắt tay (*handshake messages*) với thông điệp ứng dụng (*application messages*) [1]. Proposal và Commit thuộc nhóm thứ nhất vì chúng làm thay đổi hoặc chuẩn bị thay đổi trạng thái nhóm. Ngược lại, application messages là các tin nhắn ứng dụng được mã hóa bằng khóa sinh ra từ trạng thái của epoch hiện tại.

Trong mạng thực tế, thông điệp ứng dụng có thể đến muộn hoặc đến sai thứ tự. MLS có cơ chế hỗ trợ một mức độ nhất định cho các thông điệp bị trễ, chẳng hạn thông qua bộ khóa của người gửi (*sender ratchet*) và bộ đếm thế hệ (*generation counter*). Tuy nhiên, để bảo vệ Forward Secrecy, thiết bị không nên giữ các khóa cũ vô thời hạn. Điều này tạo ra một đánh đổi quen thuộc: nếu xóa khóa cũ quá sớm thì thông điệp đến muộn có thể không giải mã được; nếu giữ khóa cũ quá lâu thì phạm vi thiệt hại khi thiết bị bị thỏa hiệp sẽ tăng [1].

MLS cũng hỗ trợ cơ chế gia nhập từ bên ngoài, thường được gọi là External

Commit hoặc External Join [1]. Có thể hiểu đây là cách để một client chưa ở trong trạng thái hiện tại của nhóm gia nhập lại nhóm khi có đủ thông tin công khai cần thiết. Cơ chế này quan trọng đối với đồ án vì khi trạng thái đã rẽ nhánh, một thiết bị có thể cần từ bỏ nhánh cũ và tham gia lại nhánh đang được chọn.

2.4 Nền tảng mạng ngang hàng và thứ tự trong hệ phân tán

2.4.1 Đặc tính của mạng ngang hàng

Mạng ngang hàng khác với mô hình Client-Server ở chỗ không có một máy chủ trung tâm duy nhất điều phối toàn bộ luồng truyền thông. Các thiết bị có thể tự phát hiện nhau, thiết lập kết nối trực tiếp hoặc gián tiếp và trao đổi dữ liệu qua nhiều đường truyền khác nhau. Cách tổ chức này giúp hệ thống giảm phụ thuộc vào hạ tầng trung tâm, nhưng đồng thời làm cho việc điều phối trở nên khó hơn.

Trong môi trường P2P, các hiện tượng như thiết bị ngoại tuyến, độ trễ không đồng đều, trùng lặp thông điệp, biến động nút mạng hoặc phân mảnh mạng là điều bình thường. Với ứng dụng thông thường, chúng chủ yếu gây chậm hoặc mất dữ liệu tạm thời. Với MLS, các hiện tượng đó còn có thể kéo theo hệ quả nghiêm trọng hơn: một số thiết bị đã chuyển sang epoch mới trong khi thiết bị khác vẫn ở epoch cũ, hoặc hai phân vùng mạng cùng tạo ra những Commit khác nhau. Điều này cho thấy lớp mạng P2P chỉ giải quyết việc truyền thông, chứ không tự giải quyết bài toán nhất quán trạng thái mật mã.

2.4.2 go-libp2p và GossipSub

Trong nguyên mẫu của đồ án, tầng mạng được xây dựng trên go-libp2p [10]. Đây là thư viện P2P mô-đun hóa, cho phép mỗi thiết bị có một định danh mạng riêng, thiết lập kết nối qua nhiều cơ chế truyền dẫn và trao đổi dữ liệu trên nhiều luồng logic. Những khả năng này phù hợp với yêu cầu của một ứng dụng nhắn tin không phụ thuộc hoàn toàn vào máy chủ trung tâm.

Một thành phần quan trọng của libp2p là GossipSub, tức cơ chế phát tán thông điệp theo mô hình công bố/đăng ký nhận (*publish/subscribe*). Có thể hiểu đơn giản rằng khi một thiết bị gửi thông điệp vào một chủ đề chung, thông điệp đó sẽ được lan truyền dần qua các thiết bị lân cận trong mạng. Cơ chế này phù hợp để phát tán proposal, commit hoặc các thông báo trạng thái trong môi trường P2P.

Tuy nhiên, GossipSub chỉ là cơ chế phát tán thông điệp, không phải cơ chế đồng thuận. Nó không bảo đảm mọi thiết bị nhận thông điệp theo cùng một thứ tự, không tự chọn Commit thắng cuộc khi có xung đột, và cũng không bảo đảm một thông điệp chỉ được nhận đúng một lần. Vì vậy, chỉ đưa MLS lên trên GossipSub là chưa đủ để bảo vệ chuỗi trạng thái của nhóm.

2.4.3 Đồng hồ logic và thứ tự hiển thị thông điệp ứng dụng

Trong cùng một epoch, nhiều thành viên có thể gửi thông điệp ứng dụng song song. Những thông điệp này không nhất thiết cần một thứ tự mật mã toàn cục như Commit, nhưng giao diện chat vẫn cần một thứ tự hiển thị đủ ổn định giữa các thiết bị. Nếu chỉ dựa vào đồng hồ vật lý, hệ thống dễ gặp sai lệch do lệch giờ hoặc do môi trường mạng không ổn định.

Vì vậy, hệ phân tán thường sử dụng các dạng đồng hồ logic để biểu diễn quan hệ thứ tự giữa các sự kiện. Lamport Clock cho một thứ tự logic đơn giản [11]. Hybrid Logical Clock (HLC) kết hợp thời gian vật lý và bộ đếm logic để tạo ra dấu thời gian gần với thời gian thực hơn, nhưng vẫn ổn định hơn khi đồng hồ giữa các thiết bị không hoàn toàn khớp nhau [12]. Trong đồ án, loại dấu thời gian này chỉ có ý nghĩa đối với việc sắp xếp hiển thị các thông điệp ứng dụng; nó không thay thế epoch, chữ ký hay các cơ chế bảo vệ trạng thái của MLS.

2.4.4 Khoảng trống cần giải quyết

Từ các phân tích trên có thể thấy bài toán của đề tài nằm ở giao điểm giữa mật mã học nhóm và hệ phân tán. MLS cung cấp lõi mật mã mạnh nhưng giả định hệ thống triển khai phải giải quyết việc phân phối và ordering của thông điệp. Libp2p và GossipSub cung cấp hạ tầng truyền thông ngang hàng nhưng không cung cấp thứ tự tuyến tính cho Commit. HLC hỗ trợ sắp xếp thông điệp ứng dụng nhưng không bảo vệ trạng thái mật mã của nhóm [2], [10], [12].

Khoảng trống nghiên cứu vì vậy không nằm ở chỗ thiếu một giao thức mã hóa nhóm, mà nằm ở chỗ thiếu một cơ chế điều phối đủ nhẹ để phù hợp với P2P, nhưng vẫn đủ chặt để giảm nguy cơ rẽ nhánh và hỗ trợ hội tụ trở lại sau khi mạng bất ổn. Khoảng trống đó là cơ sở trực tiếp cho giải pháp sẽ được trình bày ở Chương 3.

Chương 2 đã làm rõ ngữ cảnh lý thuyết của bài toán vận hành MLS trên mạng ngang hàng. Trước hết, chương chỉ ra rằng khó khăn trung tâm của đề tài không chỉ là truyền tin trong môi trường P2P, mà là duy trì một trạng thái mật mã chung cho toàn nhóm khi không còn điểm tuân tự hóa trung tâm. Tiếp theo, chương khảo sát các hướng tiếp cận liên quan, từ Sender Keys, Megolm đến MLS trong mô hình có Delivery Service và các nỗ lực phi tập trung hóa MLS, qua đó làm nổi bật khoảng trống nghiên cứu mà đồ án hướng tới.

Bên cạnh đó, chương cũng trình bày các kiến thức nền tảng cần thiết để hiểu phần phương pháp ở các chương sau, bao gồm epoch, proposal, commit, TreeKEM, các dấu hiệu nhận biết trạng thái và cơ chế gia nhập lại nhóm. Cuối cùng, chương phân tích ngắn gọn các đặc tính của mạng ngang hàng, vai trò của

libp2p, GossipSub và đồng hồ logic trong việc hỗ trợ truyền thông và sắp xếp thông điệp ứng dụng. Những nội dung này tạo tiền đề trực tiếp cho Chương 3, nơi giải pháp điều phối phi tập trung của đề án sẽ được trình bày chi tiết.

CHƯƠNG 3. PHƯƠNG PHÁP ĐỀ XUẤT

Chương 2 đã làm rõ nền tảng của MLS, TreeKEM, mạng ngang hàng và hiện tượng phân nhánh trạng thái mật mã khi nhiều Commit cạnh tranh xuất hiện trong cùng một Epoch. Trên cơ sở đó, Chương 3 trình bày giao thức điều phối phi tập trung được đề xuất nhằm hỗ trợ MLS vận hành trên mạng ngang hàng mà không cần một Delivery Service trung tâm làm bộ tuần tự hóa.

Nội dung chương được tổ chức thành các phần chính sau. Trước hết, chương mô tả tổng quan giải pháp, kiến trúc hai lớp Go–Rust và luồng hoạt động của hệ thống. Tiếp theo, chương trình bày ba cơ chế điều phối cốt lõi gồm người ghi duy nhất theo epoch, kiểm tra nhất quán epoch và hàn gắn phân nhánh trạng thái khi mạng bị chia cắt. Sau đó, chương giới thiệu cách sử dụng đồng hồ logic lai để sắp xếp thông điệp ứng dụng, rồi khép lại bằng mô hình quản lý trạng thái giữa Go Host, Rust Sidecar và lưu trữ cục bộ bằng SQLite.

3.1 Tổng quan giải pháp

3.1.1 Bài toán cần giải quyết

Trong mô hình MLS thông thường, Delivery Service đóng vai trò quan trọng trong việc phân phối thông điệp và hỗ trợ sắp xếp thứ tự các Commit. Khi hai thành viên cùng tạo Commit tại cùng một Epoch, một DS nhất quán mạnh có thể quyết định Commit nào được chấp nhận trước, từ đó giúp toàn nhóm duy trì cùng một chuỗi Epoch tuyến tính. Khi loại bỏ DS trong môi trường P2P, hệ thống mất đi điểm tuần tự hóa trung tâm này [1], [2].

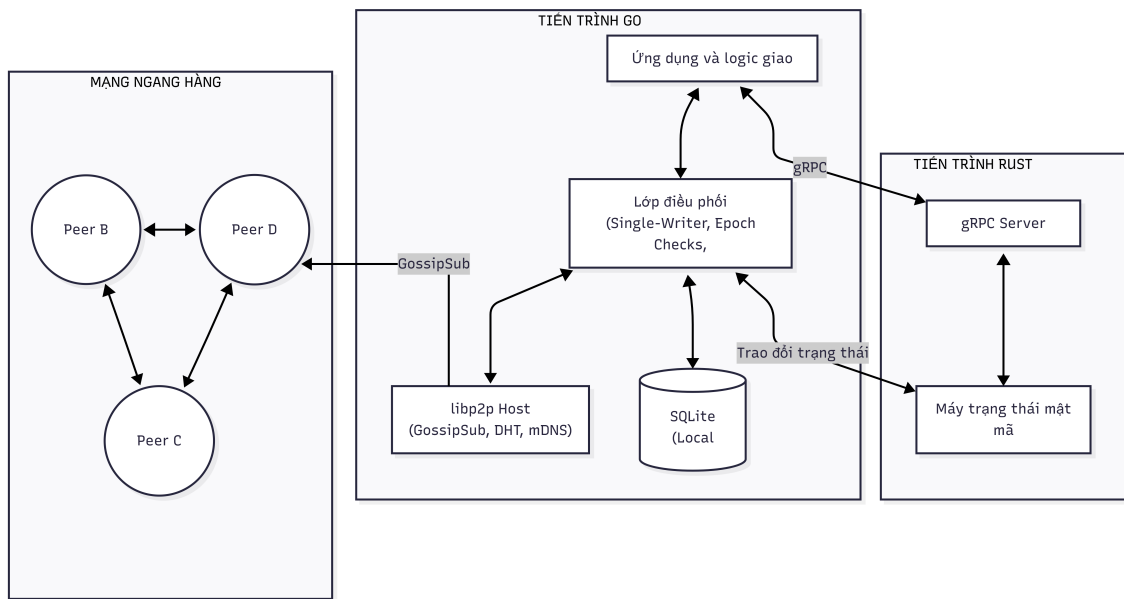
Giả sử nhóm đang ở Epoch E . Thành viên A tạo Commit C_A để chuyển nhóm sang trạng thái $S_A(E + 1)$, trong khi thành viên B đồng thời tạo Commit C_B để chuyển nhóm sang trạng thái $S_B(E + 1)$. Nếu một phần các nút nhận và áp dụng C_A trước, còn phần còn lại nhận và áp dụng C_B trước, nhóm sẽ bị tách thành hai nhánh trạng thái khác nhau. Hai nhánh này có thể cùng mang số Epoch $E + 1$, nhưng khác tree hash, transcript hash, epoch secret và khóa ứng dụng. Khi đó, các thành viên ở hai nhánh không còn chia sẻ cùng trạng thái mật mã và không thể xử lý thông điệp của nhau một cách hợp lệ [1], [5], [6].

Do đó, phương pháp đề xuất cần giải quyết bốn yêu cầu chính: (i) giảm tối đa khả năng xuất hiện concurrent commits trong điều kiện mạng liên thông; (ii) mỗi nút phải có khả năng kiểm tra thông điệp đến để tránh áp dụng sai Epoch hoặc áp dụng Commit dựa trên trạng thái không tương thích; (iii) khi phân mảnh mạng (network partition) xảy ra khiến nhiều nhánh xuất hiện, hệ thống phải tự động phát

hiện và có quy trình hội tụ trở lại một nhánh hợp lệ duy nhất; (iv) các thông điệp ứng dụng thông thường trong cùng một Epoch vẫn cần được sắp xếp hiển thị ổn định trên các thiết bị mà không phụ thuộc vào đồng hồ vật lý đồng bộ tuyệt đối.

3.1.2 Kiến trúc tổng thể

Giải pháp đề xuất được tổ chức theo kiến trúc hai lớp, gồm Go Host và Rust Sidecar. Go Host là tiến trình chính của ứng dụng, chịu trách nhiệm giao diện người dùng, lưu trữ cục bộ, mạng ngang hàng và logic điều phối. Rust Sidecar là tiến trình mật mã chuyên biệt, chịu trách nhiệm thực thi các thao tác MLS bằng thư viện OpenMLS. Sơ đồ kiến trúc triển khai chi tiết được thể hiện trong Hình 3.1.



Hình 3.1: Kiến trúc triển khai chi tiết của Go Host và Rust Sidecar

Trong kiến trúc này, Go Host là nguồn quyết định về thứ tự điều phối ở tầng ứng dụng. Rust Sidecar không tự quyết định Commit nào là hợp lệ về mặt phân tán; nó chỉ kiểm tra và xử lý thông điệp theo logic mật mã của MLS. Ngược lại, Go Host không tự triển khai mật mã MLS; nó chỉ gọi Rust Sidecar khi cần tạo Proposal, Commit, Welcome, External Commit, hoặc mã hóa/giải mã application message.

Một điểm quan trọng trong thiết kế là Rust Sidecar được giữ phi trạng thái từ góc nhìn của Go Host. Go Host lưu trạng thái nhóm bền vững trong SQLite, lấy snapshot GroupState hiện tại khi cần xử lý, gửi snapshot đó sang Rust qua gRPC, sau đó lưu lại GroupState mới do Rust trả về. Cách tổ chức này làm rõ nguồn sự thật của hệ thống và giúp quá trình khôi phục sau khi ứng dụng hoặc sidecar khởi động lại trở nên trực tiếp hơn.

3.1.3 Các thành phần chính của giao thức

Giao thức điều phối phi tập trung của đồ án gồm bốn cơ chế chính. Thứ nhất là cơ chế người ghi duy nhất theo epoch nhằm kiểm soát quyền tạo Commit. Thứ hai là cơ chế kiểm tra nhất quán epoch để bảo vệ tính đơn điệu của trạng thái cục bộ. Thứ ba là cơ chế hàn gắn phân nhánh nhằm phát hiện và xử lý sự phân kỳ trạng thái khi mạng bị chia cắt. Thứ tư là cơ chế sắp xếp thông điệp ứng dụng bằng đồng hồ logic lai HLC. Bốn cơ chế này phối hợp với nhau để hình thành lớp điều phối của hệ thống.

3.1.4 Luồng hoạt động tổng thể

Khi người dùng gửi một tin nhắn ứng dụng thông thường, Go Host lấy trạng thái nhóm hiện tại, sinh dấu thời gian HLC, gọi Rust Sidecar để mã hóa nội dung theo MLS, sau đó đóng gói bản mã vào lớp bao điều phối và phát tán qua GossipSub. Vì loại tin nhắn này không làm thay đổi tập thành viên hay trạng thái cây MLS, nó không cần đi qua cơ chế người ghi duy nhất.

Khi người dùng thực hiện thao tác thay đổi nhóm, chẳng hạn thêm thành viên, xóa thành viên hoặc cập nhật khóa, Go Host tạo Proposal và phát tán qua mạng. Mọi thành viên hợp lệ đều có thể gửi Proposal, nhưng chỉ thành viên đang nắm quyền ghi ở epoch hiện tại, gọi là Token Holder, mới được phép thu thập các Proposal hợp lệ để tạo Commit. Sau khi Commit được phát tán và được các nút áp dụng thành công, nhóm chuyển sang epoch mới, đồng thời Token Holder của epoch tiếp theo được tính lại.

Xét theo luồng logic, tiến trình Commit của hệ thống diễn ra như sau. Trước hết, một hoặc nhiều thành viên tạo Proposal tại Epoch E và phát tán các Proposal này qua mạng ngang hàng. Tiếp đó, Token Holder của Epoch E thu thập các Proposal hợp lệ từ hàng đợi cục bộ rồi gọi Rust Sidecar để tạo Commit. Commit sau khi được tạo sẽ được đóng gói trong envelope của lớp điều phối và phát tán qua GossipSub. Ở phía nhận, các nút tiến hành kiểm tra epoch, chữ ký, trạng thái nhóm và confirmation tag; nếu Commit hợp lệ, chúng cập nhật cây mật mã cục bộ, chuyển sang Epoch $E + 1$ và tự tính lại Token Holder của epoch mới bằng cùng một hàm tất định cục bộ.

Luồng trên cho thấy sự phân chia trách nhiệm giữa hai lớp của hệ thống. MLS vẫn chịu trách nhiệm về tính hợp lệ mật mã của Commit, còn lớp điều phối quyết định nút nào được quyền tạo Commit và thông điệp nào được phép đi vào lõi mật mã ở từng thời điểm.

3.2 Giao thức người ghi duy nhất theo Epoch

3.2.1 Mục tiêu thiết kế

Hiện tượng nhiều Commit cùng xuất hiện tại một epoch là nguyên nhân trực tiếp dẫn đến phân nhánh trạng thái mật mã trong MLS trên P2P. Một hướng tiếp cận là cho phép nhiều Commit xuất hiện rồi xử lý xung đột về sau. Tuy nhiên, cách này đẩy độ phức tạp vào lõi mật mã và có thể yêu cầu giữ nhiều trạng thái tạm thời, từ đó làm suy yếu phần nào thuộc tính bảo mật xuôi [5], [6]. Đồ án lựa chọn hướng ngược lại: giảm khả năng xung đột ngay từ nguồn bằng cách giới hạn quyền tạo Commit tại mỗi epoch.

Giao thức người ghi duy nhất đặt ra quy tắc sau: tại một epoch E , chỉ một thành viên trong tập ứng viên hợp lệ được quyền tạo Commit. Thành viên này được gọi là Token Holder, ký hiệu là $TH(E)$. Các thành viên khác vẫn được quyền tạo Proposal, nhưng không được tự ý tạo Commit nếu không phải là Token Holder. Cơ chế này chuyển bài toán từ chỗ nhiều nút cùng ghi vào trạng thái nhóm sang chỗ nhiều nút gửi yêu cầu nhưng chỉ một nút được quyền ghi. Trong điều kiện mạng còn liên thông và góc nhìn về tập thành viên hoạt động đủ gần nhau, các nút sẽ tính ra cùng một Token Holder, từ đó giảm mạnh khả năng xuất hiện concurrent commits mà không cần một máy chủ trung tâm.

3.2.2 Tập ứng viên hợp lệ

Token Holder không được chọn từ toàn bộ các nút đang nhìn thấy trên mạng, mà chỉ được chọn từ tập ứng viên hợp lệ của nhóm. Tập này được xác định dựa trên ba yếu tố: trạng thái thành viên theo MLS, trạng thái hoạt động quan sát được của các nút và chính sách quyền của ứng dụng. Cụ thể:

$$Eligible(E) = \mathcal{M}(E) \cap \mathcal{A}(E) \cap \mathcal{C}(E) \setminus \mathcal{S}(E) \quad (3.1)$$

Trong đó, $\mathcal{M}(E)$ là tập thành viên hợp lệ của nhóm theo trạng thái MLS tại epoch E . Tập $\mathcal{A}(E)$ là tập các thành viên được quan sát là còn hoạt động thông qua nhịp tim trạng thái hoặc các tín hiệu kết nối gần đây. Tập $\mathcal{C}(E)$ là tập các thành viên được chính sách ứng dụng cho phép tạo Commit. Cuối cùng, $\mathcal{S}(E)$ là tập các nút bị tạm loại khỏi quá trình điều phối do ngoại tuyến quá lâu, bị phát hiện lỗi hoặc bị chính sách nhóm loại trừ tạm thời.

Việc tách tập thành viên đang hoạt động khỏi tập thành viên theo MLS là cần thiết. Một thành viên có thể vẫn nằm trong Ratchet Tree nhưng đang ngoại tuyến, do đó không thể đóng vai trò Token Holder trong thực tế. Ngược lại, một nút đang trực tuyến nhưng không phải thành viên MLS của nhóm thì cũng không được tham

gia tạo Commit. Vì vậy, chỉ phần giao của các tập trên mới có ý nghĩa đối với bài toán điều phối.

3.2.3 Hàm bầu chọn Token Holder

Token Holder được chọn bằng cách tính hàm băm trên định danh nhóm, số epoch và định danh nút của từng ứng viên. Ứng viên có giá trị băm nhỏ nhất theo thứ tự từ điển được chọn làm Token Holder:

$$TH(E) = \operatorname{argmin}_{n \in Eligible(E)} H(group_id \parallel E \parallel n) \quad (3.2)$$

Trong đó H là hàm băm mật mã SHA-256. Việc đưa $group_id$ vào đầu vào của hàm băm giúp cùng một nút không nhất thiết nắm quyền ghi ở mọi nhóm. Việc đưa epoch vào hàm băm giúp quyền Token Holder thay đổi sau mỗi lần Commit, tránh tình trạng một nút luôn giữ quyền ghi quá lâu. Việc đưa định danh nút vào hàm băm giúp kết quả có tính tất định cục bộ mà không phụ thuộc vào thứ tự lưu trữ danh sách thành viên. Thuật toán bầu chọn Token Holder được mô tả trong Thuật toán 1.

Algorithm 1: Thuật toán bầu chọn Token Holder (*ComputeTokenHolder*)

Đầu vào: *groupID*: định danh nhóm; *epoch*: Epoch hiện tại; *members*: tập thành viên MLS; *activeView*: tập thành viên hoạt động; *authorized*: tập thành viên có quyền tạo Commit; *suspended*: tập thành viên bị đình chỉ.

Đầu ra : *holderID*: định danh Token Holder được chọn.

$eligible \leftarrow members \cap activeView \cap authorized \setminus suspended$;

if *eligible* là tập rỗng **then**

return Lỗi *NoEligibleCommitter*;

end

$bestID \leftarrow \text{null}$;

$bestHash \leftarrow \text{MAX_HASH}$;

for mỗi *peerID* trong *eligible* **do**

$data \leftarrow \text{Encode}(groupID, epoch, peerID)$;

$candidateHash \leftarrow \text{SHA256}(data)$;

if $candidateHash < bestHash$ **then**

$bestHash \leftarrow candidateHash$;

$bestID \leftarrow peerID$;

end

end

return *bestID*;

Thuật toán này không yêu cầu các nút trao đổi phiếu bầu. Nếu các nút có cùng trạng thái nhóm và cùng tập ứng viên hoạt động, chúng sẽ tính ra cùng một Token Holder. Do đó, bước bầu chọn không phát sinh thêm thông điệp mạng; chi phí chủ yếu nằm ở phép tính cục bộ, tăng tuyến tính theo số lượng ứng viên $O(k)$ với $k = |Eligible(E)|$.

3.2.4 Tạo Proposal và Commit

Giao thức phân biệt rõ Proposal và Commit. Proposal là yêu cầu thay đổi nhóm, còn Commit là thao tác thực sự chuyển nhóm sang Epoch mới [1]. Mọi thành viên hợp lệ đều có thể tạo Proposal, nhưng chỉ Token Holder được tạo Commit.

Khi một nút muốn thêm thành viên, xóa thành viên hoặc cập nhật khóa, nó gọi Rust Sidecar để tạo Proposal hợp lệ theo MLS, sau đó phát tán Proposal qua GossipSub. Các nút nhận Proposal sẽ kiểm tra chữ ký, kiểm tra Epoch và đưa Proposal vào hàng đợi cục bộ nếu hợp lệ. Token Holder của Epoch hiện tại định kỳ gom các Proposal hợp lệ để tạo Commit. Khoảng thời gian gom Proposal được

cấu hình bằng tham số trì hoãn *batch_delay*. Việc gom Proposal giúp giảm số lần tăng Epoch không cần thiết khi nhiều thay đổi xảy ra gần nhau và giảm khả năng Proposal bị bỏ sót [1]. Quy trình tạo Commit của Token Holder được mô tả trong Thuật toán 2.

Algorithm 2: Quy trình tạo Commit của Token Holder
(*CommitByTokenHolder*)

Đầu vào: *localState*: trạng thái MLS cục bộ tại Epoch E ; *proposalQueue*: hàng đợi Proposal đã nhận; *holderID*: Token Holder tại Epoch E ; *selfID*: định danh nút cục bộ.

Đầu ra : Bản tin Commit được phát tán nếu nút cục bộ là Token Holder.

if *selfID* $\neq$ *holderID* **then**

 Kết thúc (Không có quyền tạo Commit);

end

Chọn tập Proposal hợp lệ P từ *proposalQueue*;

if P rỗng và không có yêu cầu cập nhật khóa cục bộ **then**

 Kết thúc;

end

Gửi yêu cầu *CreateCommit*(*localState*, P) tới Rust Sidecar;

if Rust Sidecar trả về Commit hợp lệ **then**

 Đóng gói Commit vào *CoordinationEnvelope*;

 Lưu trạng thái pending commit cục bộ;

 Phát tán bản tin Commit qua *GossipSub*;

end

else

 Ghi nhận lỗi và loại bỏ các Proposal không hợp lệ khỏi hàng đợi;

end

3.2.5 Chuyển quyền và failover

Sau khi Commit hợp lệ được áp dụng, nhóm chuyển sang Epoch $E + 1$. Vì hàm bầu chọn phụ thuộc vào Epoch, Token Holder mới sẽ được tính lại tự động. Cơ chế này giúp quyền ghi được luân chuyển giữa các thành viên, tránh phụ thuộc lâu dài vào một nút.

Trong mạng P2P, Token Holder có thể ngoại tuyến hoặc mất kết nối trước khi tạo Commit. Để tránh bế tắc, mỗi nút duy trì heartbeat và *ActiveView*. Nếu Token Holder hiện tại không gửi heartbeat hoặc không tạo Commit trong khoảng thời gian vượt quá *holder_timeout*, các nút có thể đánh dấu Token Holder đó là tạm thời không hoạt động và tính lại tập *Eligible*(E). Khi tập ứng viên thay đổi, hàm

băm sẽ chọn ra Token Holder mới.

Cơ chế failover này không phải đồng thuận Byzantine đầy đủ. Trong trường hợp mạng bị phân mảnh, các phân vùng khác nhau có thể có ActiveView khác nhau và tính ra Token Holder khác nhau. Đồ án chấp nhận đây là hệ quả tự nhiên của network partition. Mục tiêu của Single-Writer không phải ngăn mọi fork trong mọi điều kiện vật lý, mà là loại bỏ concurrent commits trong điều kiện mạng liên thông và biến fork thành một tình huống ngoại lệ có thể phát hiện, sau đó xử lý bằng Fork Healing.

3.3 Kiểm tra nhất quán Epoch

3.3.1 Mục tiêu thiết kế

MLS yêu cầu mỗi Commit phải được áp dụng trên đúng trạng thái mà nó được tạo ra. Nếu một Commit được sinh từ epoch E nhưng lại bị áp dụng lên trạng thái khác, kết quả xử lý sẽ không còn đúng. Vì vậy, mỗi nút cần một lớp kiểm tra trước khi đưa thông điệp vào Rust Sidecar. Cơ chế kiểm tra nhất quán epoch có nhiệm vụ bảo vệ ranh giới giữa mạng P2P bất đồng bộ và lõi mật mã MLS. Trong môi trường này, thông điệp có thể đến muộn, đến sớm, lặp lại hoặc đến sai thứ tự. Lớp điều phối vì thế phải phân loại các thông điệp thành thông điệp hiện tại, thông điệp cũ, thông điệp tương lai hoặc thông điệp có dấu hiệu phân nhánh [1], [2].

3.3.2 Lớp bao điều phối (Coordination Envelope)

Mọi thông điệp MLS khi truyền qua P2P đều được bọc trong một lớp bao điều phối, gọi là Coordination Envelope. Lớp bao này không thay thế bản tin MLS, mà chỉ bổ sung các thông tin phụ trợ phục vụ định tuyến, kiểm tra epoch và phục hồi trạng thái. Cấu trúc khái niệm của Coordination Envelope được thể hiện trong Mã nguồn 3.1.

Listing 3.1: Cấu trúc Coordination Envelope ở lớp điều phối Go

```

1  type CoordinationEnvelope struct {
2      GroupID      []byte    // Định danh nhóm MLS
3      SenderID     PeerID    // Định danh nút gửi o tang P2P
4      MessageID    []byte    // Chong xu ly lap lai tin nhan
5      ContentType  Type      // Proposal | Commit | Application
           ↳ | Heartbeat | StateSync
6      Epoch        uint64    // Epoch tai thoi diem tao payload
7      HLCTimestamp HLC       // Dong ho logic lai HLC cho tin
           ↳ nhan ung dung
8      TreeHashHint []byte    // Tree hash cuc bo cua nguoi gui
           ↳ (tuy chon)
9      CommitHash   []byte    // Commit hash tuong ung (tuy chon
           ↳ )

```

```

10     Payload      []byte    // Du lieu ciphertext/plaintext
      ↪ MLS
11     Signature    []byte    // Chu ky xac thuc lop dieu phoi
12 }

```

Trường *group_id* xác định nhóm MLS tương ứng. Trường *sender_id* xác định nút gửi ở tầng P2P. Trường *message_id* dùng để tránh xử lý lặp lại cùng một lớp bao. Trường *content_type* giúp lớp điều phối phân biệt thông điệp bắt tay và thông điệp ứng dụng. Trường *epoch* cho biết payload được tạo ra ở phiên bản trạng thái nào của nhóm. Trường *hlc_timestamp* chỉ phục vụ việc sắp xếp thông điệp ứng dụng. Các trường như *tree_hash_hint* hoặc *commit_hash* hỗ trợ phát hiện phân nhánh và đồng bộ trạng thái. Trường *payload* chứa bản tin MLS thực sự. Cuối cùng, trường *signature* xác thực phần metadata của lớp điều phối, trong khi bản thân payload MLS vẫn được bảo vệ bởi cơ chế xác thực riêng của MLS.

3.3.3 Phân loại thông điệp theo Epoch

Khi nhận một Coordination Envelope, Go Host lấy epoch cục bộ của nhóm tương ứng và so sánh với *env.epoch*. Có ba trường hợp chính.

Thứ nhất, nếu *env.epoch = localEpoch*, thông điệp thuộc về epoch hiện tại. Trong trường hợp này, nếu các dấu hiệu trạng thái như *tree_hash_hint* hoặc *epoch_authenticator* không tương thích với trạng thái cục bộ, Go Host kích hoạt cơ chế phát hiện phân nhánh. Ngược lại, nếu đây là Proposal hoặc Commit thì payload được chuyển cho Rust Sidecar để xử lý phần bắt tay; còn nếu là thông điệp ứng dụng thì payload được chuyển cho Rust Sidecar để giải mã rồi đưa vào hàng đợi hiển thị theo HLC.

Thứ hai, nếu *env.epoch < localEpoch*, thông điệp thuộc về quá khứ. Với Commit hoặc Proposal cũ, nút cục bộ không áp dụng lại. Với thông điệp ứng dụng cũ, hệ thống chỉ thử giải mã nếu chính sách giữ khóa cũ trong một khoảng hữu hạn cho phép.

Thứ ba, nếu *env.epoch > localEpoch*, thông điệp thuộc về tương lai, cho thấy nút cục bộ đã bỏ lỡ một hoặc nhiều Commit trung gian. Khi đó, Go Host đưa envelope vào vùng đệm thông điệp tương lai và kích hoạt quy trình đồng bộ trạng thái còn thiếu.

Quy trình xử lý này được mô tả trong Thuật toán 3.

Algorithm 3: Quy trình phân loại và xử lý Envelope đến
(*ProcessIncomingEnvelope*)

Đầu vào: *env*: bản tin CoordinationEnvelope nhận được; *localState*: trạng thái nhóm cục bộ.

Đầu ra : Hành động xử lý tương ứng cục bộ.

if *env.message_id* đã được xử lý **then**

 Bỏ qua envelope;

 Kết thúc;

end

if *env.groupID* không tồn tại cục bộ **then**

 Bỏ qua hoặc đưa vào hàng đợi xử lý gia nhập;

 Kết thúc;

end

localEpoch $\leftarrow$ *localState.epoch*;

if *env.epoch* == *localEpoch* **then**

if *env.tree_hash_hint* $\neq$ *localState.tree_hash* **then**

 Kích hoạt *ForkDetection(env)*;

 Kết thúc;

end

if *env.contentType* là *Proposal* hoặc *Commit* **then**

 Chuyển *env.payload* sang Rust Sidecar để xử lý handshake;

end

if *env.contentType* là *Application* **then**

 Chuyển *env.payload* sang Rust Sidecar để giải mã;

 Đưa tin nhắn giải mã vào hàng đợi hiển thị theo HLC;

end

end

else if *env.epoch* < *localEpoch* **then**

if *env.contentType* là *Application* và thuộc *stale window* **then**

 Thử giải mã bằng khóa cũ được lưu giữ;

end

else if *env.contentType* là *Proposal* **then**

 Gửi yêu cầu *RePropose* cho người gửi hoặc bỏ qua;

end

else

 Bỏ qua envelope;

end

end

else

3.3.4 Đồng bộ trạng thái khi gặp thông điệp tương lai

Khi một nút nhận thông điệp tương lai, nó không tự ý tăng epoch cục bộ. Epoch chỉ được tăng khi Rust Sidecar xử lý thành công một Commit hợp lệ. Do đó, quy trình đồng bộ trạng thái phải lấy được chuỗi Commit còn thiếu thay vì nhảy cóc trạng thái.

Quy trình đồng bộ trạng thái hoạt động theo hai chế độ. Ở chế độ thứ nhất, nút yêu cầu các bản tin Commit còn thiếu từ epoch hiện tại đến epoch đích. Nếu các Commit này còn được lưu trong lịch sử và có thể kiểm tra tuần tự, nút áp dụng từng Commit để bắt kịp trạng thái. Ở chế độ thứ hai, nếu lịch sử bị khuyết thiếu đáng kể hoặc trạng thái đã rẽ nhánh phức tạp, nút yêu cầu thông tin nhóm đầy đủ như *GroupInfo* hoặc *Welcome*, hoặc thực hiện quy trình gia nhập lại từ bên ngoài. Việc không tự động áp dụng thông điệp tương lai giúp bảo vệ tính toàn vẹn của máy trạng thái mật mã [1].

3.3.5 Xử lý Proposal bị bỏ sót

Một dị thường quan trọng trong môi trường P2P là Proposal bị bỏ sót do Commit đã xảy ra trước khi Proposal đến được Token Holder. Ví dụ, tại Epoch E , nút A gửi Proposal P_A , nút B gửi Proposal P_B . Token Holder nhận P_A trước, tạo Commit đưa nhóm lên $E + 1$. Sau đó P_B mới đến và trở thành Proposal cũ.

Để tránh làm mất thao tác của người dùng, giao thức sử dụng cơ chế đề xuất lại, gọi là Re-Propose. Khi một nút phát hiện Proposal của mình không được đưa vào Commit của epoch tương ứng, nó kiểm tra xem thao tác đó còn hợp lệ trong epoch mới hay không. Nếu còn hợp lệ, nút tạo lại Proposal trên trạng thái mới và phát tán lại. Nếu không còn hợp lệ, ứng dụng hiển thị thông báo lỗi rõ ràng cho người dùng. Quy trình này được mô tả trong Thuật toán 4.

Algorithm 4: Quy trình Re-Propose cho Proposal bị bỏ sót (*ReProposeIfDropped*)

Đầu vào: *localProposal*: Proposal do nút cục bộ tạo tại Epoch E ;
appliedCommit: Commit đã được áp dụng để chuyển sang Epoch $E + 1$; *newState*: trạng thái nhóm sau Commit.

if *localProposal* nam trong *appliedCommit* **then**

Đánh dấu Proposal đã hoàn thành và áp dụng;
 Kết thúc;

end

if *localProposal* không con hợp lệ trong *newState* **then**

Đánh dấu Proposal thất bại và thông báo cho người dùng;
 Kết thúc;

end

Tạo Proposal mới tương đương tại Epoch $E + 1$;

Phát tán Proposal mới qua GossipSub;

3.4 Cơ chế phục hồi phân nhánh trạng thái

3.4.1 Bối cảnh network partition

Single-Writer hoạt động tốt khi các nút có cùng góc nhìn về tập thành viên đang hoạt động. Tuy nhiên, trong mạng P2P, phân mảnh mạng (network partition) có thể chia nhóm thành nhiều phân vùng không thể liên lạc với nhau. Mỗi phân vùng khi đó có thể loại các nút ngoài phân vùng khỏi góc nhìn hoạt động cục bộ và tính ra Token Holder riêng. Kết quả là nhiều phân vùng có thể tạo các Commit khác nhau dựa trên cùng một epoch gốc. Đây là giới hạn nền tảng của hệ phân tán bất đồng bộ. Vấn đề quan trọng là khi mạng được kết nối lại, hệ thống phải phát hiện được phân nhánh và đưa nhóm hội tụ trở lại một trạng thái duy nhất [6], [9].

3.4.2 Phát hiện fork bằng heartbeat trạng thái

Mỗi nút định kỳ phát một thông điệp thông báo trạng thái nhóm, gọi là *GroupStateAnnouncement*, cho các nhóm mà nó tham gia qua kênh GossipSub. Thông điệp này không chứa bí mật mật mã, mà chỉ mang các định danh trạng thái công khai cần thiết cho việc đối chiếu. Cấu trúc của thông điệp được thể hiện trong Mã nguồn 3.2.

Listing 3.2: Cấu trúc thông điệp State Announcement để phát hiện Fork

```
1 type GroupStateAnnouncement struct {
2     GroupID          []byte // Định danh nhóm
3     SenderID         PeerID  // Định danh nút gửi o tang
```

```

    ↪ P2P
4   Epoch          uint64      // Epoch cuc bo hien tai cua
    ↪ nut
5   TreeHash       []byte      // Tree hash tuong ung cua cay
    ↪ TreeKEM
6   EpochAuthenticator []byte  // Gia tri xac thuc Epoch hien
    ↪ tai
7   LastCommitHash []byte      // Ma bam Commit cuoi cung
    ↪ duoc ap dung
8   ActiveMemberCount uint32    // So thanh vien dang hoat
    ↪ dong trong ActiveView
9   HLCTimestamp   HLC         // Thoi gian logic lai HLC
10  Signature       []byte      // Chu ky xac thuc cua nguoi
    ↪ gui
11 }

```

Khi nhận được thông điệp này, nút cục bộ so sánh các dấu hiệu trạng thái với trạng thái của chính mình. Nếu định danh nhóm giống nhau nhưng *tree_hash* hoặc *epoch_authenticator* khác nhau tại cùng một epoch, hệ thống kết luận rằng hai nút không còn ở cùng trạng thái mật mã và kích hoạt cơ chế Fork Healing. Việc so sánh các giá trị này giúp phát hiện hiệu quả những trường hợp trạng thái đã rẽ nhánh [1], [6].

3.4.3 Hàm trọng số nhánh

Sau khi phát hiện fork, các nút cần chọn một nhánh làm nhánh thắng cuộc để hội tụ. Đồ án đề xuất sử dụng hàm trọng số nhánh có thứ tự ưu tiên từ cao xuống thấp:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (3.3)$$

Trong biểu thức trên, $C_{active}(B)$ là số thành viên đang hoạt động và cùng xác nhận nhánh B ; $E(B)$ là epoch hiện tại của nhánh; còn $H_{commit}(B)$ là mã băm của Commit cuối cùng trên nhánh đó, được dùng làm tiêu chí phá hòa tất định khi các tiêu chí trước chưa đủ để phân biệt.

Hai nhánh được so sánh theo thứ tự từ trái sang phải. Nhánh có nhiều thành viên hoạt động hơn được ưu tiên nhằm giảm thiểu số lượng thiết bị phải thực hiện gia nhập lại và bảo vệ trải nghiệm của đa số thành viên đang online. Nếu bằng nhau, nhánh có Epoch cao hơn được chọn vì phản ánh mức tiến hóa trạng thái lớn hơn. Nếu vẫn bằng nhau, mã băm Commit được so sánh từ điển để phá hòa một cách tất định cục bộ. Quy trình so sánh được mô tả trong Thuật toán 5.

Algorithm 5: Thuật toán so sánh trọng số nhánh để giải quyết Fork
(*CompareBranchWeight*)

Đầu vào: *localBranch*: thông tin nhánh cục bộ; *remoteBranch*: thông tin nhánh nhận từ mạng.

Đầu ra : Kết quả so sánh: *BranchLocal* (Nhánh cục bộ thắng), *BranchRemote* (Nhánh từ mạng thắng) hoặc *BranchEqual* (Bằng nhau).

```

if localBranch.tree_hash == remoteBranch.tree_hash and
    localBranch.epoch_authenticator ==
    remoteBranch.epoch_authenticator then
    |   return BranchEqual;
end
if localBranch.active_count ≠ remoteBranch.active_count then
    |   if localBranch.active_count > remoteBranch.active_count then
    |   |   return BranchLocal;
    |   end
    |   else
    |   |   return BranchRemote;
    |   end
end
if localBranch.epoch ≠ remoteBranch.epoch then
    |   if localBranch.epoch > remoteBranch.epoch then
    |   |   return BranchLocal;
    |   end
    |   else
    |   |   return BranchRemote;
    |   end
end
if localBranch.last_commit_hash < remoteBranch.last_commit_hash
    then
    |   return BranchLocal;
end
else
    |   return BranchRemote;
end

```

Hàm trọng số này là một chính sách hội tụ tắt định trong bối cảnh các nút trung

thực nhưng bị phân mảnh mạng. Đối với các kịch bản có nút cố tình giả mạo thông tin, hệ thống vẫn cần bổ sung thêm các cơ chế xác thực quyền hạn và các chính sách bảo mật khác.

3.4.4 Quy trình Fork Healing

Sau khi xác định nhánh thắng, các nút thuộc nhánh thua thực hiện quy trình hàn gắn theo bốn bước logic liên tiếp. Thứ nhất, nút đóng băng nhánh thua, tức dừng tạo Commit và dừng gửi các tin nhắn ứng dụng mới trên trạng thái đó; những tin nhắn cục bộ chưa gửi được đưa vào hàng đợi phục hồi. Thứ hai, các bí mật mật mã không còn thuộc nhánh được chọn phải được xóa sạch khỏi bộ nhớ nhằm bảo toàn tính bảo mật xuôi. Thứ ba, nút lấy thông tin nhóm từ nhánh thắng, chẳng hạn *GroupInfo* hoặc *Welcome*, xác thực các thông tin cần thiết và thực hiện quy trình gia nhập lại thông qua cơ chế *External Join* của MLS. Thứ tư, nếu chính sách ứng dụng cho phép phục hồi, các tin nhắn ứng dụng do chính nút đó tạo trong thời gian phân mảnh mạng nhưng chưa xuất hiện trên nhánh thắng sẽ được mã hóa lại và phát lại dưới epoch mới của nhánh thắng. Nút tuyệt đối không phát lại tin nhắn của nút khác nhằm bảo toàn tính xác thực của nguồn gửi [1].

Quy trình Fork Healing tổng quát được mô tả trong Thuật toán 6.

Algorithm 6: Quy trình hàn gắn phân nhánh mật mã (*ForkHealing*)

Đầu vào: *localBranch*: thông tin nhánh cục bộ; *winningBranch*: thông tin nhánh thắng cuộc; *outbox*: hàng đợi tin nhắn cục bộ chưa gửi trong thời gian phân mảnh.

if *localBranch* == *winningBranch* **then**

 Tiếp tục hoạt động bình thường;
 Kết thúc;

end

Phong tỏa các thao tác ghi và gửi tin nhắn trên *localBranch*;

Lưu danh sách tin nhắn do chính mình tạo trong *outbox*;

Tiêu hủy tức thời các bí mật mật mã liên quan đến *localBranch*;

Lấy thông tin nhóm (*GroupInfo*) mới nhất từ *winningBranch*;

Xác thực danh tính và tính hợp lệ của *winningBranch*;

Gửi yêu cầu *ExternalJoin* tới Rust Sidecar để gia nhập nhánh thắng cuộc;

if Gia nhập thành công **then**

 Cập nhật trạng thái MLS cục bộ sang nhánh mới;

for mỗi *msg* trong *outbox* do chính mình tạo **do**

 Mã hóa lại *msg* dưới Epoch mới của nhóm;
 Phát tán tin nhắn qua GossipSub;

end

end

else

 Đánh dấu trạng thái lỗi và yêu cầu can thiệp đồng bộ thủ công;

end

3.4.5 Lưu trạng thái bền vững trong quá trình hàn gắn

Fork Healing là quy trình có nhiều bước và có thể bị gián đoạn bởi sự cố thiết bị hoặc mất điện. Để bảo đảm tính bền vững, Go Host lưu trạng thái phục hồi vào SQLite dưới dạng một thực thể trạng thái, được minh họa trong Mã nguồn 3.3.

Listing 3.3: Cấu trúc HealingState lưu trữ trong cơ sở dữ liệu SQLite

```

1  type HealingState struct {
2      GroupID          []byte    // Định danh nhóm MLS
3      OldBranchID      []byte    // Nhánh cũ bị có lỗi
4      WinningBranchID  []byte    // Nhánh thắng cuộc được chọn
5      ↪ de hoi tu
6      Phase            string    // Detecting | Freezing |
7      ↪ Joining | Replaying | Completed | Failed

```

```

6      PendingMessages  [][]byte // Hàng đợi tin nhắn cục bộ cần
      ↪ replay
7      LastError        string    // Ghi nhận lỗi hệ thống nếu có
8      UpdatedAt        time.Time // Thời gian cập nhật trạng
      ↪ thái gần nhất
9  }

```

Khi ứng dụng khởi động lại, Go Host kiểm tra bảng trạng thái phục hồi này. Nếu ghi nhận nhóm đang ở pha *Joining* hoặc *Replaying*, hệ thống sẽ tự động tiếp tục quy trình từ bước tương ứng thay vì bắt đầu lại từ đầu, đảm bảo tính sẵn sàng và an toàn cho dữ liệu người dùng.

3.5 Thứ tự thông điệp ứng dụng bằng Hybrid Logical Clock

3.5.1 Lý do cần HLC

Epoch trong MLS chỉ phản ánh thứ tự thay đổi trạng thái mật mã (handshake). Nó không cung cấp thứ tự toàn phần cho các tin nhắn ứng dụng được gửi song song trong cùng một Epoch. Alice và Bob có thể gửi tin nhắn đồng thời và các nút khác nhau nhận được chúng theo thứ tự ngược nhau do độ trễ truyền dẫn mạng. Nếu ứng dụng hiển thị theo thời điểm nhận cục bộ, người dùng trên các thiết bị khác nhau sẽ quan sát thấy lịch sử hội thoại không nhất quán.

Sử dụng đồng hồ vật lý cục bộ không khả thi trong mạng P2P vì các thiết bị thường bị lệch giờ hoặc không đồng bộ NTP. Do đó, đề án sử dụng đồng hồ logic lai HLC để gán nhãn thời gian cho các thông điệp ứng dụng. HLC kết hợp thời gian vật lý và bộ đếm logic, biểu diễn dưới dạng bộ ba [12]:

$$HLC = (L, C, N) \quad (3.4)$$

Trong đó L là thời gian vật lý lớn nhất từng được ghi nhận, C là bộ đếm logic cho các sự kiện đồng thời, và N là định danh nút (PeerID) để phá hòa một cách tất định.

3.5.2 Tạo timestamp khi gửi tin

Khi người dùng gửi tin nhắn ứng dụng, Go Host gọi hàm `HLC_Now`. Nếu thời gian vật lý hiện tại của hệ thống lớn hơn giá trị L đang lưu, HLC đặt L bằng thời gian vật lý hiện tại và khởi tạo bộ đếm $C = 0$. Ngược lại, nếu thời gian vật lý không tăng, hệ thống tăng bộ đếm C lên 1 để bảo đảm tính đơn điệu của thời gian. Quy trình được mô tả trong Thuật toán 7.

Algorithm 7: Thuật toán sinh thời gian logic lai (HLC_Now)

Đầu vào: hlc : trạng thái HLC cục bộ; $nodeID$: định danh nút cục bộ.

Đầu ra : Timestamp HLC mới ($L, C, nodeID$).

$pt \leftarrow$ thời gian vật lý hiện tại (mili-giây);

if $pt > hlc.L$ **then**

$hlc.L \leftarrow pt$;

$hlc.C \leftarrow 0$;

end

else

$hlc.C \leftarrow hlc.C + 1$;

end

return ($hlc.L, hlc.C, nodeID$);

Dấu thời gian này được đóng gói vào Coordination Envelope ở phần metadata bên ngoài payload của bản tin.

3.5.3 Cập nhật HLC khi nhận tin

Khi nhận một envelope có chứa timestamp HLC từ mạng, nút cục bộ tiến hành cập nhật HLC của mình dựa trên thời gian vật lý cục bộ và HLC nhận được để đảm bảo đồng hồ logic tiến lên một cách nhất quán. Quy trình cập nhật được mô tả trong Thuật toán 8.

Algorithm 8: Thuật toán cập nhật đồng hồ logic lai (HLC_Update)

Đầu vào: hlc : trạng thái HLC cục bộ; msg : timestamp HLC nhận được
 $(msg.L, msg.C, msg.NodeID)$.

Đầu ra : Trạng thái HLC cục bộ sau khi cập nhật.

$pt \leftarrow$ thời gian vật lý hiện tại (mili-giây);

if $msg.L - pt > MAX_DRIFT_MS$ **then**

Đánh dấu thông điệp nghi vấn clock drift lớn và xử lý theo chính sách;
 Kết thúc;

end

$newL \leftarrow \max(hlc.L, msg.L, pt)$;

if $newL == hlc.L$ **and** $newL == msg.L$ **then**

$hlc.C \leftarrow \max(hlc.C, msg.C) + 1$;

end

else if $newL == hlc.L$ **then**

$hlc.C \leftarrow hlc.C + 1$;

end

else if $newL == msg.L$ **then**

$hlc.C \leftarrow msg.C + 1$;

end

else

$hlc.C \leftarrow 0$;

end

$hlc.L \leftarrow newL$;

return hlc ;

Hằng số MAX_DRIFT_MS được sử dụng để phát hiện các thông điệp có timestamp sai lệch đáng kể so với đồng hồ cục bộ nhằm ngăn chặn hành vi cố tình đẩy timestamp về tương lai để làm xáo trộn thứ tự dòng chat.

3.5.4 Thứ tự hiển thị tất định

Tin nhắn ứng dụng sau khi giải mã được sắp xếp hiển thị dựa trên khóa thứ tự:

$$OrderKey = (epoch, HLC.L, HLC.C, senderID, messageID) \quad (3.5)$$

Quy tắc sắp xếp tuân theo: trước hết phân loại theo Epoch của nhóm mật mã; trong cùng Epoch, sắp xếp theo thời gian logic lai L và bộ đếm C ; nếu trùng, sử dụng định danh người gửi và định danh tin nhắn để phá hòa một cách tất định. Nhờ đó, tất cả các thiết bị trong nhóm nhận cùng một tập tin nhắn sẽ hiển thị hội thoại

theo một thứ tự hoàn toàn đồng nhất.

3.6 Quản lý trạng thái Go-Rust và lưu trữ cục bộ

3.6.1 Lý do sử dụng Rust Sidecar phi trạng thái

Trong nguyên mẫu hiện tại, Go Host lưu toàn bộ trạng thái MLS bền vững trong SQLite. Mỗi lần cần mã hóa, giải mã hoặc tạo Commit, Go Host lấy trạng thái nhóm hiện tại, gửi sang Rust Sidecar qua gRPC và nhận lại trạng thái mới sau khi OpenMLS xử lý. Cách tổ chức này có chi phí tuần tự hóa và truyền dữ liệu qua kênh liên tiến trình, nhưng đổi lại giúp trách nhiệm lưu trữ và khôi phục trạng thái được phân định rõ ràng.

Do đó, Rust Sidecar được thiết kế như một lớp xử lý mật mã thuần túy: Rust không quyết định ordering, không lưu trạng thái nhóm lâu dài và không trở thành nguồn sự thật của ứng dụng. Mọi quyết định về epoch, Token Holder, lưu lịch sử và phục hồi sau sự cố đều nằm ở Go Host cùng cơ sở dữ liệu cục bộ.

3.6.2 Phân chia trách nhiệm giữa SQLite và Rust Sidecar

SQLite đóng vai trò là kho lưu trữ dữ liệu bền vững của Go Host, đảm bảo dữ liệu không bị mất khi ứng dụng tắt hoặc thiết bị mất điện. SQLite lưu trữ lịch sử tin nhắn, trạng thái điều phối, hàng đợi tin nhắn chưa gửi và snapshot trạng thái MLS. Rust Sidecar nhận snapshot này như dữ liệu đầu vào, thực hiện thao tác OpenMLS tương ứng, rồi trả lại kết quả để Go Host lưu xuống SQLite.

3.6.3 Luồng xử lý khi gửi tin nhắn ứng dụng

Quy trình gửi tin nhắn ứng dụng được mô tả chi tiết trong Thuật toán 9.

Algorithm 9: Quy trình gửi tin nhắn ứng dụng (*SendApplicationMessage*)

Đầu vào: *groupID*: định danh nhóm; *plaintext*: nội dung tin nhắn dạng rõ.

localState $\leftarrow$ lấy trạng thái nhóm cục bộ từ SQLite;

timestamp $\leftarrow$ *HLC_Now()*;

Gọi Rust Sidecar:

EncryptApplicationMessage(localState.groupState, plaintext);

Nhận về bản mã *mlsCiphertext* và trạng thái nhóm mới từ Sidecar;

Lưu trạng thái nhóm mới vào SQLite;

Tạo *CoordinationEnvelope* mang *groupID*, *epoch* = *localState.epoch*,

hlc_timestamp = *timestamp* và *payload* = *mlsCiphertext*;

Lưu tin nhắn vào SQLite ở trạng thái chờ gửi (*pending*);

Phát tán envelope qua GossipSub mạng lưới P2P;

Tin nhắn ứng dụng không làm thay đổi Epoch mật mã nên không yêu cầu quyền

Token Holder, nhưng vẫn phải mang Epoch hiện tại để nút nhận xác định đúng khóa giải mã.

3.6.4 Luồng xử lý khi thay đổi thành viên

Quy trình xử lý khi người dùng muốn thêm hoặc xóa thành viên được mô tả trong Thuật toán 10.

Algorithm 10: Quy trình yêu cầu thay đổi thành viên
(*RequestMembershipChange*)

Đầu vào: *groupID*: định danh nhóm; *operation*: loại thao tác
(Add/Remove); *targetMember*: thành viên mục tiêu.

Gọi Rust Sidecar để tạo đối tượng Proposal tương ứng với thao tác;
Tạo *CoordinationEnvelope* mang loại *Proposal*, *epoch = localEpoch* và
payload = proposal;
Lưu Proposal vào hàng đợi cục bộ *proposalQueue*;
Phát tán Proposal qua GossipSub mạng P2P;
if Bản thân nút cục bộ là Token Holder của Epoch hiện tại **then**
 Thiết lập bộ đếm thời gian gửi Commit sau khoảng trì hoãn
 batch_delay;
end

Việc tách biệt Proposal và Commit cho phép nhiều thành viên đề xuất thay đổi cùng lúc mà không phá vỡ nguyên tắc một người ghi duy nhất (Single-Writer) tại mỗi Epoch.

3.7 Tính đúng đắn trực giác của phương pháp

Tính đúng đắn trực giác của giao thức điều phối đề xuất có thể được lý giải qua bốn luận điểm chính. Trước hết, trong điều kiện mạng liên thông, cơ chế Single-Writer làm giảm mạnh khả năng xuất hiện concurrent commits vì tại mỗi Epoch chỉ Token Holder mới được phép sinh bản tin Commit để nâng trạng thái nhóm. Tiếp theo, lớp Epoch Checks hoạt động như một bộ lọc biên, ngăn các thông điệp đến sai thứ tự hoặc không đồng bộ đi thẳng vào lõi MLS, từ đó giúp máy trạng thái cục bộ tiếp tục tiến hóa theo chuỗi Epoch đơn điệu và tránh những lỗi do xử lý Commit ngoài luồng.

Trong trường hợp mạng bị phân mảnh, các phân vùng có thể tiến hóa độc lập và tạo ra những nhánh trạng thái khác nhau. Khi đó, quy trình Fork Healing cùng hàm trọng số nhánh tất định cho phép hệ thống lựa chọn một nhánh tiếp tục và đưa các nút ở nhánh còn lại gia nhập lại nhánh được chọn thông qua cơ chế hợp lệ của MLS. Cách xử lý này không loại bỏ hoàn toàn mọi hệ quả của network partition,

nhưng giúp hệ thống có cơ chế hội tụ an toàn sau khi kết nối được khôi phục, đồng thời hạn chế việc kéo dài vòng đời của vật liệu khóa thuộc nhánh cũ.

Cuối cùng, đối với các thông điệp ứng dụng được gửi song song trong cùng một Epoch, đồng hồ logic lai HLC cung cấp một quy tắc sắp xếp hiển thị ổn định giữa các thiết bị. Nhờ đó, lớp ứng dụng có thể duy trì một lịch sử hội thoại nhất quán hơn về mặt quan sát mà không can thiệp vào tiến trình mật mã của MLS.

3.8 Kết chương

Chương 3 đã trình bày chi tiết giao thức điều phối phi tập trung được đề xuất cho MLS trên mạng ngang hàng. Trên cơ sở kiến trúc hai lớp Go–Rust Sidecar, chương đã mô tả bốn cơ chế cốt lõi của giải pháp, gồm cơ chế người ghi duy nhất để kiểm soát quyền tạo Commit, cơ chế kiểm tra nhất quán epoch để bảo vệ trạng thái cục bộ, cơ chế hàn gắn phân nhánh để hỗ trợ hội tụ sau khi mạng bị chia cắt, và cơ chế sắp xếp thông điệp ứng dụng bằng HLC. Các cơ chế này kết hợp với nhau để hình thành một hướng tiếp cận không phụ thuộc vào Delivery Service tập trung, đồng thời tránh phải sử dụng các thuật toán đồng thuận nặng cho mọi Commit.

Những đề xuất thiết kế và các thuật toán logic trình bày trong chương này là cơ sở trực tiếp để Chương 4 tiến hành phân tích lý thuyết sâu hơn về tính nhất quán trạng thái mật mã, chi phí tính toán, chi phí truyền thông và phạm vi bảo vệ an toàn thông tin của hệ thống.

CHƯƠNG 4. PHÂN TÍCH LÝ THUYẾT

4.1 Mục tiêu và phạm vi phân tích

Chương này tập trung phân tích cơ sở lý thuyết của lớp điều phối phi tập trung đã được đề xuất ở Chương 3. Mục tiêu của chương là làm rõ vì sao các cơ chế Single-Writer, Epoch Checks, Fork Healing và HLC Message Ordering có thể hỗ trợ MLS vận hành trong môi trường mạng ngang hàng mà không cần một Delivery Service trung tâm đảm nhiệm việc điều phối Commit.

Trong MLS, trạng thái nhóm tiến hóa theo chuỗi các epoch và mỗi Commit đưa nhóm sang một epoch mới [1]. Khi tồn tại Delivery Service nhất quán mạnh, hệ thống có thể dựa vào thành phần này để áp đặt một thứ tự tuyến tính cho các Commit và xử lý các tình huống cạnh tranh [2]. Ngược lại, trong môi trường P2P bất đồng bộ, các client có thể nhận thông điệp theo những thứ tự khác nhau và phải tự hòa giải trạng thái. Bài toán lý thuyết của đề án vì thế có thể phát biểu ngắn gọn như sau: làm thế nào để các nút vẫn duy trì được một chuỗi trạng thái MLS hợp lệ, hoặc ít nhất có thể phát hiện và phục hồi khi chuỗi trạng thái đó bị phân nhánh.

Do đặc tính của hệ phân tán, nhất là khi xảy ra network partition, hệ thống không thể đồng thời bảo đảm tuyệt đối tính nhất quán mạnh, tính sẵn sàng và khả năng chịu phân mảnh [9]. Vì vậy, mục tiêu của đề án được giới hạn theo ba hướng: giảm khả năng sinh concurrent commits khi các nút có cùng góc nhìn về nhóm, phát hiện được fork khi các phân vùng tiến hóa sang các trạng thái MLS khác nhau, và đưa các nút hội tụ trở lại một trạng thái MLS hợp lệ sau khi mạng được nối lại.

Các phân tích trong chương này được trình bày theo dạng bất biến và lập luận tính đúng đắn. Đây không phải là chứng minh hình thức đầy đủ theo mô hình toán học hoặc mô hình đối kháng mạnh, nhưng đủ để làm rõ vì sao các cơ chế đề xuất phù hợp với mục tiêu của đề án và vì sao chúng không đòi hỏi sửa đổi các thuật toán mật mã cốt lõi của MLS.

4.2 Mô hình phân tích

Xét một nhóm MLS có định danh $groupID$ và tập thành viên tại epoch E là M_E . Mỗi nút trong nhóm duy trì một trạng thái cục bộ S_E , trong đó phần cốt lõi là trạng thái MLS ở tầng mật mã và phần còn lại là metadata điều phối ở tầng ứng dụng. Trong phạm vi chương này, cần phân biệt hai khái niệm. Thứ nhất là trạng thái MLS đầy đủ, bao gồm toàn bộ thông tin mà OpenMLS dùng để xử lý Proposal, Commit và application message. Thứ hai là các dấu hiệu trạng thái mà lớp điều phối sử dụng để quan sát và nhận diện phân nhánh. Ở góc độ phân tích của đề án,

các dấu hiệu trọng tâm cho mục đích điều phối là số epoch, tree hash và định danh của Commit gần nhất.

Mạng P2P được xem là môi trường bất đồng bộ. Thông điệp có thể đến trễ, đến sai thứ tự, bị lặp hoặc tạm thời không đến được. Trong điều kiện mạng liên thông, các nút cuối cùng có thể nhận được thông tin từ nhau, nhưng không giả định rằng mọi nút nhận thông điệp theo cùng một thứ tự. Trong điều kiện network partition, nhóm có thể bị chia thành nhiều phân vùng, mỗi phân vùng chỉ quan sát được một phần các thành viên đang hoạt động.

Ký hiệu $ActiveView_i(E)$ là tập thành viên mà nút i quan sát là đang hoạt động tại epoch E . Trong điều kiện mạng ổn định, các nút có xu hướng hội tụ về cùng một $ActiveView$. Ngược lại, khi mạng bị phân mảnh, hai phân vùng khác nhau có thể có hai $ActiveView$ khác nhau. Đây là nguyên nhân trực tiếp khiến cùng một công thức bầu Token Holder có thể cho ra kết quả khác nhau giữa các phân vùng.

Trong phạm vi phân tích, đồ án giả định các nút tuân thủ giao thức ở mức cơ bản và OpenMLS xử lý đúng các thao tác mật mã. Các hành vi đối kháng phức tạp không phải trọng tâm của chương này. Cách giới hạn như vậy giúp phần phân tích tập trung đúng vào bài toán chính của đồ án, tức là duy trì và phục hồi tính nhất quán trạng thái MLS trong môi trường P2P không ổn định, thay vì mở rộng sang toàn bộ không gian đe dọa của một giao thức phân tán hoàn chỉnh.

4.3 Bất biến 1: Epoch cục bộ không được thoái lui

4.3.1 Phát biểu bất biến

Tại mỗi nút, giá trị epoch cục bộ chỉ được giữ nguyên hoặc tăng lên sau khi nút xử lý thành công một Commit hợp lệ. Nút không được áp dụng một thông điệp thuộc epoch cũ lên trạng thái hiện tại, vì điều này có thể dẫn đến rollback trạng thái mật mã hoặc xử lý bản mã trên một trạng thái cây MLS không còn tương thích.

Bất biến này có thể viết ngắn gọn như sau:

$$E_i(t+1) \geq E_i(t)$$

trong đó $E_i(t)$ là epoch cục bộ của nút i tại thời điểm logic t .

4.3.2 Cơ sở từ MLS

MLS tổ chức lịch sử nhóm thành một chuỗi các epoch, trong đó mỗi epoch biểu diễn một trạng thái nhóm với tập thành viên đã xác thực cùng chia sẻ trạng thái mật mã [1]. Mỗi Commit khởi tạo một epoch mới bằng cách yêu cầu các thành viên áp dụng một tập Proposal và cập nhật ratchet tree [1]. Vì vậy, nếu một nút áp dụng

thông điệp sai epoch, nút đó có nguy cơ đặt một thông điệp hợp lệ về mặt định dạng lên một trạng thái mật mã không còn tương thích.

4.3.3 Cơ chế bảo toàn bất biến

Cơ chế Epoch Checks của đồ án bọc mỗi thông điệp nhận từ mạng trong một lớp envelope chứa *groupID*, loại thông điệp và epoch của người gửi. Khi nhận thông điệp, lớp điều phối so sánh epoch của thông điệp với epoch cục bộ trước khi chuyển thông điệp xuống OpenMLS.

Quan hệ epoch	Hành động của lớp điều phối
$E_{msg} = E_{local}$	Thông điệp được chuyển tiếp cho lớp MLS xử lý theo luồng tiến hóa bình thường.
$E_{msg} < E_{local}$	Thông điệp bị từ chối áp dụng lên trạng thái MLS hiện tại nhằm ngăn chặn hành vi thoái lui (rollback) trạng thái.
$E_{msg} > E_{local}$	Thông điệp chưa được xử lý ngay; nút đưa vào bộ đệm tạm thời và kích hoạt quá trình đồng bộ trạng thái.

Bảng 4.1: Quy tắc xử lý thông điệp theo epoch

Mệnh đề 1. Nếu một nút chỉ tăng epoch sau khi xử lý thành công Commit hợp lệ và từ chối áp dụng thông điệp thuộc epoch cũ, thì epoch cục bộ của nút là đơn điệu không giảm.

Lập luận. Giả sử một nút chỉ tăng epoch sau khi xử lý thành công một Commit hợp lệ. Theo quy tắc định tuyến tại Bảng 4.1, nút từ chối áp dụng mọi thông điệp có $E_{msg} < E_{local}$, do đó trạng thái MLS cục bộ không bao giờ bị ghi đè hoặc thoái lui về quá khứ. Đối với thông điệp có $E_{msg} > E_{local}$, nút buộc phải đồng bộ hóa chuỗi trạng thái thay vì áp dụng trực tiếp, đảm bảo mọi quá trình chuyển đổi trạng thái đều phải tuân tự trải qua các Commit hợp lệ. Do OpenMLS chỉ sinh trạng thái mới thông qua quá trình xử lý Commit hợp lệ, kết hợp với cơ chế kiểm soát vòng ngoài của lớp điều phối, chuỗi giá trị epoch cục bộ tại mọi nút luôn thỏa mãn tính chất đơn điệu không giảm.

Bất biến này không đảm bảo rằng mọi nút luôn có cùng epoch tại mọi thời điểm. Trong mạng P2P, một số nút có thể tạm thời đi sau do mất kết nối hoặc nhận thông điệp chậm. Điều bất biến đảm bảo là mỗi nút không tự rollback trạng thái MLS của chính nó.

4.4 Bất biến 2: Với cùng một view, chỉ có một Token Holder

4.4.1 Phát biểu bất biến

Tại epoch E , nếu hai nút có cùng $groupID$, cùng số epoch và cùng tập ứng viên hợp lệ, chúng phải tính ra cùng một Token Holder. Token Holder là nút duy nhất được quyền tạo Commit cho epoch đó trong view tương ứng.

Tập ứng viên hợp lệ được xác định như sau:

$$\begin{aligned} Eligible(E) = & GroupMembers(E) \cap ActiveView(E) \\ & \cap AuthorizedCommitters(E) \setminus RemovedOrSuspended(E) \end{aligned} \quad (4.1)$$

Trong đó, $GroupMembers(E)$ là tập thành viên có trong trạng thái MLS tại epoch E ; $ActiveView(E)$ là tập thành viên đang được quan sát là còn hoạt động; $AuthorizedCommitters(E)$ là tập thành viên được chính sách ứng dụng cho phép tạo Commit; và $RemovedOrSuspended(E)$ là tập thành viên tạm thời không được xét làm Token Holder.

Token Holder được tính bằng quy tắc tất định:

$$TokenHolder(E) = \operatorname{argmin}_{n \in Eligible(E)} H(groupID \parallel E \parallel n) \quad (4.2)$$

Trong đó H là hàm băm mật mã, ví dụ SHA-256.

Mệnh đề 2. Nếu hai nút có cùng $groupID$, cùng epoch E và cùng tập $Eligible(E)$, thì chúng chọn cùng một Token Holder.

Lập luận. Nếu hai nút i và j có cùng $groupID$, cùng epoch E và cùng tập $Eligible(E)$, thì đầu vào của Công thức 4.2 trên hai nút là giống nhau. Vì hàm băm là tất định, tập giá trị băm thu được trên hai nút cũng giống nhau. Do cùng áp dụng phép chọn argmin trên cùng một tập giá trị, hai nút sẽ chọn cùng một Token Holder.

Trong điều kiện các nút có cùng view, Single-Writer đảm bảo chỉ có một Token Holder, do đó hệ thống không cần thêm một vòng bỏ phiếu mạng để thống nhất ai là người tạo Commit. Mỗi nút có thể tự tính cục bộ và đi đến cùng kết quả. Các nút không phải Token Holder vẫn có thể tạo Proposal, nhưng không trực tiếp tạo Commit tại epoch đó.

4.4.2 Ý nghĩa đối với concurrent commits

Trong MLS, Commit là thao tác làm thay đổi trạng thái nhóm và đưa nhóm sang epoch mới [1]. Với DS nhất quán mạnh, DS có thể phá hòa khi hai thành viên gửi Commit cùng lúc [2]. Khi không có DS trung tâm, nếu mọi thành viên đều được tự do Commit, hệ thống dễ sinh nhiều Commit cạnh tranh tại cùng một epoch. Single-Writer giải quyết vấn đề này bằng cách giới hạn quyền Commit vào một Token Holder duy nhất trong mỗi view.

Cơ chế này không khẳng định hệ thống không bao giờ fork. Khi network partition xảy ra, các phân vùng có thể có *ActiveView* khác nhau, dẫn tới *Eligible(E)* khác nhau và Token Holder khác nhau. Vì vậy, Single-Writer không phải cơ chế chống fork tuyệt đối trong mọi điều kiện mạng. Vai trò đúng của nó là hạn chế concurrent commits trong điều kiện các nút có cùng góc nhìn, từ đó giảm đáng kể nguy cơ fork khi mạng còn liên thông.

4.5 Bất biến 3: Fork trạng thái phải phát hiện được

4.5.1 Hiện tượng fork trạng thái mật mã

Một fork trạng thái mật mã xảy ra khi hai tập nút có cùng *groupID* nhưng tiến hóa sang hai trạng thái MLS khác nhau. Trường hợp dễ gây nhầm lẫn nhất là hai nhánh đều mang cùng một số epoch, nhưng lại không còn dẫn về cùng một chuỗi Commit và cùng một trạng thái cây MLS. Khi đó, dù số epoch giống nhau, hai nhánh thực chất không còn chia sẻ cùng một MLS group state.

DMLS cũng chỉ ra vấn đề tương tự: trong môi trường phi tập trung, có thể xuất hiện nhiều Commit cho cùng một số epoch, khiến số nguyên epoch không còn đủ để định danh duy nhất một trạng thái nhóm [6]. Điều này củng cố nhận định rằng trong P2P MLS, chỉ kiểm tra số epoch là chưa đủ; hệ thống cần so sánh thêm các dấu hiệu trạng thái khác để biết liệu hai nút có đang đứng trên cùng một nhánh hay không.

4.5.2 Fingerprint trạng thái

MLS định nghĩa nhiều giá trị có thể dùng để nhận diện trạng thái nhóm. Chẳng hạn, tree hash tóm tắt trạng thái ratchet tree và được dùng trong GroupContext để phản ánh cấu trúc cây hiện tại [1]. Ngoài ra, ở mức lý thuyết còn có những giá trị khác như transcript hash hay epoch authenticator. Trong phạm vi lớp điều phối mà đề án tập trung phân tích, việc nhận diện nhánh được xây dựng trên một tập dấu hiệu ngắn gọn hơn, đủ để phục vụ mục tiêu phát hiện fork và chọn nhánh phục hồi.

Dựa trên các dấu hiệu mà lớp điều phối thực sự quan sát được, các nút trao đổi định kỳ một bản tóm tắt trạng thái:

$$StateSummary = (groupID, E, treeHash, \\ commitHash)$$

Trong đó *commitHash* là giá trị băm của Commit gần nhất mà lớp điều phối gắn với nhánh đang xét. Giá trị này không thay thế toàn bộ trạng thái MLS, nhưng là một fingerprint phụ trợ hữu ích để phân biệt các nhánh và phá hòa một cách tất định khi cần chọn nhánh tiếp tục.

Mệnh đề 3. Khi network partition xảy ra, hệ thống có thể phát hiện sự khác biệt giữa các nhánh bằng cách so sánh các dấu hiệu trạng thái mà lớp điều phối quan sát được, từ đó làm cơ sở cho quá trình phục hồi sau khi các phân vùng kết nối trở lại.

Lập luận. Giả sử hai nút i và j có cùng *groupID*. Nếu chúng cùng ở epoch E nhưng có $treeHash_i \neq treeHash_j$, hoặc nếu chúng công bố hai giá trị *commitHash* khác nhau cho trạng thái hiện tại, thì lớp điều phối có cơ sở để kết luận rằng hai nút không còn đứng trên cùng một nhánh tiến hóa của nhóm. Nói cách khác, chỉ riêng số epoch không đủ để nhận diện duy nhất trạng thái nhóm; cần kết hợp thêm các dấu hiệu phản ánh cây MLS và lịch sử Commit gần nhất.

Lập luận quan trọng ở đây là fork MLS không phải là một trạng thái mơ hồ ở tầng ứng dụng. Khi hai nhánh đã khác nhau về trạng thái cây hoặc khác nhau về Commit gần nhất đang được gắn với trạng thái hiện hành, sự phân kỳ đó đã để lại dấu vết đủ rõ để lớp điều phối nhận ra. Vì vậy, hệ thống không cần trao đổi toàn bộ lịch sử thông điệp để phát hiện mọi trường hợp; nó chỉ cần so sánh một tập dấu hiệu trạng thái đủ ngắn gọn nhưng vẫn có ý nghĩa.

4.6 Bất biến 4: Không hợp nhất trực tiếp hai trạng thái MLS đã phân nhánh

4.6.1 Vấn đề của việc gộp trạng thái

Khi hai nhánh MLS đã phân kỳ, mỗi nhánh có lịch sử Commit, ratchet tree, transcript và key schedule riêng. Việc cố gắng trộn trực tiếp khóa, cây hoặc transcript của hai nhánh để tạo ra một trạng thái chung mới là không phù hợp với mô hình của MLS. Trong MLS, trạng thái nhóm chỉ nên tiến hóa thông qua các thao tác hợp lệ như Proposal và Commit; mỗi Commit đưa nhóm sang một epoch mới và cập nhật các thành phần liên quan của trạng thái theo đúng quy tắc giao thức [1]. Nếu lớp điều phối tự tạo ra một trạng thái mật mã mới bằng cách gộp thủ công hai nhánh đã fork, trạng thái đó không còn là kết quả của một chuỗi Commit hợp lệ. Khi đó, hệ thống không còn có thể dựa trực tiếp vào các bảo đảm của

MLS/OpenMLS.

Do đó, đề án đặt ra bất biến sau: khi fork đã xảy ra, hệ thống không hợp nhất trực tiếp hai trạng thái MLS. Thay vào đó, hệ thống chọn một nhánh tiếp tục và đưa các nút ở nhánh còn lại gia nhập lại nhóm để tạo ra một trạng thái MLS hợp lệ mới.

4.6.2 Quy tắc chọn nhánh

Giả sử sau khi network partition kết thúc, hệ thống quan sát được tập các nhánh:

$$\mathcal{B} = \{B_1, B_2, \dots, B_k\}$$

Mỗi nhánh B được gán với một trọng số:

$$W(B) = (C_{active}(B), E(B), H_{commit}(B)) \quad (4.3)$$

Trong đó $C_{active}(B)$ là số thành viên đang hoạt động của nhánh, $E(B)$ là epoch hiện tại của nhánh, và $H_{commit}(B)$ là giá trị băm của Commit gần nhất gắn với nhánh đó. Trọng số này được so sánh theo thứ tự từ điển từ trái sang phải: ưu tiên nhánh có nhiều thành viên hoạt động hơn; nếu bằng nhau thì ưu tiên nhánh có epoch cao hơn; nếu vẫn bằng nhau thì dùng H_{commit} như tie-breaker tất định.

Nhánh thắng được chọn bằng một hàm tất định:

$$B_{win} = SelectBranch(\mathcal{B}) = \operatorname{argmax}_{B \in \mathcal{B}} W(B) \quad (4.4)$$

Việc so sánh được hiểu là so sánh từ điển theo thứ tự các thành phần trong $W(B)$.

Mệnh đề 4. Nếu các nút cuối cùng quan sát được cùng một tập nhánh và cùng áp dụng quy tắc lựa chọn đã nêu, chúng sẽ hội tụ về cùng một nhánh thắng để phục hồi sau fork.

Lập luận. Hàm $SelectBranch$ là một quy tắc tất định trên cùng một đầu vào quan sát. Vì vậy, khi hai nút đã biết cùng tập nhánh $\mathcal{B}$ và gán cho mỗi nhánh cùng một trọng số $W(B)$, kết quả lựa chọn của chúng bắt buộc trùng nhau. Ý nghĩa của mệnh đề này không nằm ở chỗ một hàm tất định cho cùng đầu vào sẽ cho cùng đầu ra, mà ở chỗ cơ chế phục hồi của đề án không cần thêm một thành phần phá hòa

trung tâm. Điều kiện cần là thông tin về các nhánh phải đủ để các nút hình thành cùng một bức tranh quan sát; trước thời điểm đó, các quyết định cục bộ khác nhau vẫn có thể tạm thời xuất hiện.

Sau khi nhánh thắng được xác định, các nút thuộc nhánh thua không tiếp tục gửi thông điệp mới bằng trạng thái cũ. Chúng đóng băng trạng thái cũ ở tầng ứng dụng, sau đó gia nhập lại nhánh thắng bằng cơ chế phù hợp của MLS, chẳng hạn Welcome hoặc External Commit tùy điều kiện triển khai. RFC 9420 mô tả External Commit như một cơ chế cho phép một thành viên bên ngoài tham gia hoặc resync vào nhóm trong những điều kiện nhất định [1].

Suy ra, sau quá trình healing, hệ thống không duy trì song song nhiều nhánh cho các thông điệp mới. Thay vào đó, các nút hội tụ về một trạng thái MLS hợp lệ được tạo ra thông qua cơ chế của MLS, không phải thông qua việc tự trộn khóa cũ.

4.6.3 Giới hạn của lập luận

Lập luận trên phụ thuộc vào hai điều kiện. Thứ nhất, các phân vùng mạng phải kết nối trở lại để thông tin về các nhánh được trao đổi. Nếu partition kéo dài vô hạn, không có cơ chế P2P nào có thể buộc các phân vùng không liên lạc với nhau phải hội tụ tức thời. Thứ hai, các nút cần dần quan sát được cùng tập nhánh hoặc ít nhất cùng nhánh có trọng số cao nhất. Trước khi thông tin hội tụ, một số nút có thể tạm thời chưa chọn cùng kết quả.

Những giới hạn này không làm mất ý nghĩa của Fork Healing. Chúng chỉ phản ánh bản chất của hệ phân tán: khi mất liên lạc, hệ thống không thể vừa cho phép các phân vùng tiếp tục hoạt động cục bộ, vừa buộc mọi phân vùng luôn có cùng trạng thái ở mọi thời điểm.

4.7 Xử lý thông điệp phát sinh trong nhánh thua

Trong thời gian network partition, người dùng ở mỗi phân vùng có thể tiếp tục gửi application message. Khi partition kết thúc và một nhánh được chọn làm nhánh tiếp tục, các thông điệp phát sinh ở nhánh thua đặt ra một vấn đề riêng: chúng là dữ liệu ứng dụng hợp lệ đối với người gửi, nhưng bản mã của chúng được tạo dưới một trạng thái MLS không còn được tiếp tục sử dụng.

Vì vậy, đồ án không phát lại nguyên trạng các bản mã cũ của nhánh thua. Nếu chính sách ứng dụng cho phép phục hồi nội dung, nút chỉ phát lại các thông điệp do chính nó tạo ra và mã hóa lại nội dung dưới epoch hợp lệ của nhánh thắng. Cách làm này có ba ý nghĩa.

Thứ nhất, hệ thống không kéo dài vòng đời sử dụng của khóa cũ thuộc nhánh thua. Thứ hai, mỗi nút chỉ có thể phát lại nội dung mà nó là tác giả, tránh việc một

nút tự ý đại diện cho nút khác để tái tạo thông điệp. Thứ ba, thông điệp sau khi được phát lại sẽ là application message mới, được mã hóa và xác thực trong trạng thái MLS hiện hành của nhánh thắng.

Lập luận này không khẳng định rằng mọi dữ liệu phát sinh trong partition đều có thể được phục hồi trọn vẹn. Một số thông điệp có thể bị mất nếu tác giả không còn lưu plaintext cục bộ hoặc chính sách ứng dụng không cho phép replay. Tuy nhiên, cách xử lý này giữ được ranh giới quan trọng: phục hồi dữ liệu ứng dụng không được đánh đổi bằng việc hợp nhất hoặc tái sử dụng tùy tiện trạng thái mật mã cũ.

4.8 Bất biến 5: HLC chỉ sắp xếp thông điệp ứng dụng, không quyết định Commit

4.8.1 Phân biệt crypto ordering và application ordering

Trong hệ thống, cần phân biệt hai loại thứ tự khác nhau. Thứ tự thứ nhất là thứ tự Commit, quyết định chuỗi trạng thái mật mã của MLS. Thứ tự này ảnh hưởng trực tiếp đến epoch, ratchet tree và key schedule. Thứ tự thứ hai là thứ tự hiển thị application message trong cùng một epoch. Thứ tự này phục vụ trải nghiệm người dùng và đồng bộ dữ liệu ứng dụng.

MLS đã có cơ chế riêng để xử lý application message bị trễ hoặc đến sai thứ tự thông qua group identifier, epoch và generation counter [1]. Tuy nhiên, trong một ứng dụng P2P, các nút vẫn cần một quy tắc ổn định để hiển thị các thông điệp đã giải mã hợp lệ theo cùng một thứ tự tương đối. Đây là vai trò của HLC trong đồ án.

4.8.2 Cơ sở của HLC

Hybrid Logical Clock kết hợp thông tin thời gian vật lý và đồng hồ logic. Theo Kulkarni và Demirbas, HLC bảo toàn được một tính chất quan trọng: nếu sự kiện e xảy ra trước sự kiện f theo quan hệ happened-before, thì timestamp HLC của e sẽ nhỏ hơn timestamp HLC của f [12]. Trong đồ án này, tính chất đó được khai thác ở mức sắp xếp thông điệp ứng dụng sau khi thông điệp đã vượt qua các bước kiểm tra và giải mã hợp lệ.

Trong đồ án, timestamp của một application message được biểu diễn dưới dạng:

$$HLC = (L, C, NodeID) \quad (4.5)$$

Trong đó L là thành phần thời gian logic gắn với thời gian vật lý, C là bộ đếm logic dùng khi nhiều sự kiện có cùng L , và $NodeID$ là khóa phá hòa tất định.

Mệnh đề 5. HLC tạo thứ tự hiển thị tất định cho cùng một tập thông điệp đã

được giải mã hợp lệ, không thay thế cơ chế ordering của Commit.

Lập luận. Khi một nút gửi application message, nó gắn HLC hiện tại vào message. Khi một nút nhận message từ nút khác, nó cập nhật HLC cục bộ dựa trên timestamp nhận được và thời gian cục bộ. Nếu message A là nguyên nhân dẫn đến message B , chẳng hạn người dùng gửi phản hồi sau khi đã đọc A , thì HLC của B sẽ lớn hơn HLC của A theo quan hệ từ điển. Điều này giúp hệ thống giữ được thứ tự nhân quả cơ bản trong quá trình hiển thị mà không trao cho HLC vai trò quyết định trạng thái mật mã.

Đối với các message đồng thời, tức không có quan hệ nhân quả rõ ràng, hệ thống sử dụng so sánh từ điển trên $(L, C, NodeID)$ để tạo thứ tự hiển thị tất định. Nhờ đó, khi hai nút đã có cùng một tập message hợp lệ, chúng có thể sắp xếp tập message đó theo cùng một quy tắc mà không cần bổ sung một cơ chế ordering riêng ở tầng ứng dụng.

4.8.3 Giới hạn của HLC trong đồ án

HLC không được dùng để quyết định Commit nào hợp lệ. HLC cũng không được dùng để sửa fork trạng thái MLS. Một Commit chỉ hợp lệ khi được MLS/OpenMLS xác thực và phù hợp với trạng thái epoch hiện tại. Vì vậy, HLC chỉ nằm ở tầng ứng dụng, sau các bước xác thực và giải mã, nhằm phục vụ thứ tự hiển thị và đồng bộ hội thoại.

Sự phân định này đóng vai trò thiết yếu. Nếu nhầm HLC với cơ chế ordering cho Commit, hệ thống có thể vô tình cho phép một timestamp ứng dụng ảnh hưởng đến tiến trình key schedule của MLS. Đồ án tránh điều này bằng cách tách rõ hai tầng: Commit ordering thuộc lớp điều phối MLS, còn application ordering thuộc lớp hiển thị và đồng bộ dữ liệu.

4.9 Phân tích chi phí lý thuyết

Phần này phân tích chi phí của hệ thống theo ba lớp: chi phí mật mã MLS, chi phí điều phối P2P và chi phí triển khai. Việc tách ba lớp là cần thiết vì độ phức tạp thuật toán của MLS không phản ánh đầy đủ độ trễ end-to-end của một hệ thống thực tế còn có lưu trữ cục bộ, truyền thông P2P và lớp phối hợp vận hành.

4.9.1 Chi phí gửi application message

Trong mô hình mã hóa pairwise, người gửi thường phải tạo hoặc bọc khóa riêng cho từng người nhận. Với nhóm có N thành viên, chi phí này có xu hướng tăng tuyến tính theo số người nhận.

Với MLS, application message được bảo vệ bằng khóa dẫn xuất từ trạng thái nhóm tại epoch hiện tại. Người gửi không cần tạo một bản mã độc lập cho từng

thành viên. Vì vậy, xét riêng thao tác mã hóa nội dung ứng dụng, chi phí không tăng tuyến tính theo số lượng thành viên như mô hình pairwise. Đây là một trong những lý do MLS phù hợp hơn cho truyền thông nhóm quy mô lớn [1].

Tuy nhiên, trong hệ thống P2P, chi phí toàn trình còn phụ thuộc vào lớp mạng. Nếu một thông điệp phải được truyền trực tiếp tới nhiều peer, chi phí mạng có thể tăng theo số lượng peer hoặc theo cơ chế lan truyền của overlay. Do đó, cần phân biệt giữa chi phí mật mã của MLS và chi phí phân phối message của hệ thống P2P.

4.9.2 Chi phí cập nhật thành viên và rekey

Các thao tác thêm thành viên, loại bỏ thành viên hoặc cập nhật khóa đều được biểu diễn thông qua Proposal và Commit. Ở mức TreeKEM, ratchet tree cho phép cập nhật khóa hiệu quả hơn so với việc mã hóa riêng cho từng thành viên; RFC 9420 mô tả rằng việc mã hóa tới các subtree giúp giảm chi phí từ tuyến tính xuống theo chiều cao cây trong một số thao tác, ví dụ loại bỏ một thành viên khỏi nhóm [1].

Trong triển khai thực tế, chi phí Commit không chỉ gồm phần mật mã lõi. Nó còn bao gồm chi phí tạo Proposal, tạo Commit, xử lý Welcome hoặc External Commit, lưu trạng thái mới, truyền message qua mạng P2P và đồng bộ các nút nhận. Vì vậy, ở Chương 5, việc đánh giá cần tách hai loại số liệu: chi phí MLS core và chi phí end-to-end của toàn hệ thống.

4.9.3 Chi phí của Single-Writer

Cơ chế Single-Writer không yêu cầu một vòng đồng thuận mạng riêng để bầu Token Holder trong điều kiện các nút có cùng view. Mỗi nút tự tính Token Holder bằng cách duyệt qua tập $Eligible(E)$ và áp dụng hàm băm cho từng ứng viên. Nếu số thành viên hợp lệ là N , chi phí tính toán cục bộ của bước chọn Token Holder là $O(N)$ trong cách cài đặt trực tiếp.

Điểm quan trọng là chi phí này là chi phí cục bộ, không phải chi phí nhiều vòng mạng. Trong bối cảnh P2P, tránh thêm các vòng đồng thuận cho mỗi Commit là một lợi thế quan trọng, vì độ trễ mạng và peer churn có thể làm các giao thức đồng thuận trở nên nặng hơn nhu cầu thực tế của ứng dụng.

4.9.4 Chi phí của Epoch Checks và Fork Detection

Epoch Checks có chi phí nhỏ vì mỗi message chỉ cần so sánh một số trường metadata như *groupID*, epoch và loại thông điệp trước khi chuyển xuống MLS. Chi phí này có thể xem là $O(1)$ trên mỗi message nếu không tính phần xử lý MLS bên dưới.

Fork Detection yêu cầu các nút trao đổi một bản tóm tắt trạng thái chứa số epoch,

tree hash và commit hash gần nhất. Vì số trường metadata trong bản tóm tắt này là hằng số, kích thước của một announcement đơn lẻ có thể xem là $O(1)$ theo kích thước nhóm. Ở quy mô toàn hệ thống, chi phí mạng của cơ chế này còn chịu ảnh hưởng bởi tần suất announcement và số lượng kết nối mà mỗi nút duy trì trong overlay P2P.

4.9.5 Chi phí của Fork Healing

Fork Healing phát sinh khi hệ thống đã có phân nhánh trạng thái. Chi phí của quá trình này gồm ba phần: phát hiện nhánh, chọn nhánh, và đưa các nút ở nhánh thua gia nhập lại. Phần chọn nhánh có chi phí phụ thuộc vào số nhánh quan sát được k , thường nhỏ hơn nhiều so với số thành viên nhóm. Phần tốn kém hơn là quá trình gia nhập lại, vì nó cần thực hiện các thao tác MLS như Welcome hoặc External Commit và đồng bộ trạng thái mới.

Về lý thuyết, Fork Healing là chi phí bất thường, không phải chi phí trên mọi message. Nó chỉ xuất hiện khi có partition hoặc fork. Do đó, đề án chấp nhận chi phí này để đổi lấy khả năng duy trì hoạt động cục bộ trong thời gian mạng bị chia cắt và khả năng hội tụ sau khi kết nối được khôi phục.

4.9.6 Ảnh hưởng của kiến trúc sidecar phi trạng thái

Trong kiến trúc triển khai của đề án, Go Host lưu snapshot trạng thái nhóm trong SQLite và truyền GroupState sang Rust sidecar khi cần thực hiện thao tác MLS. Rust xử lý snapshot này bằng OpenMLS rồi trả lại trạng thái mới để Go Host lưu bền vững. Cách thiết kế này không làm thay đổi độ phức tạp lý thuyết của TreeKEM, nhưng làm phát sinh thêm chi phí tuần tự hóa, truyền dữ liệu qua gRPC và ghi lại trạng thái khi quy mô nhóm tăng.

Do đó, khi đánh giá hiệu năng, cần tránh nhầm lẫn giữa hai tầng. Nếu đo MLS core, kết quả phản ánh chi phí mật mã và xử lý trạng thái trong OpenMLS. Nếu đo end-to-end, kết quả phản ánh thêm chi phí IPC, lưu trữ, truyền mạng và điều phối. Trong phạm vi đề án, các kết quả thực nghiệm chủ yếu dùng để kiểm chứng tính đúng đắn, khả năng hội tụ và thuộc tính bảo mật; tối ưu hóa chi phí IPC và persistence được xem là hướng phát triển tiếp theo.

4.10 Tổng hợp các bất biến và cơ chế bảo toàn

Bảng 4.2 tổng hợp các bất biến chính được phân tích trong chương này và cơ chế tương ứng trong thiết kế của đề án.

Qua các bất biến trên, có thể thấy đóng góp cốt lõi của đề án không nằm ở việc thay đổi MLS, mà nằm ở việc xây dựng một lớp điều phối giúp MLS thích nghi với môi trường P2P. Lớp này thay thế một phần vai trò ordering thường do DS đảm

Bất biến / Thuộc tính	Cơ chế bảo toàn	Ý nghĩa
Epoch cục bộ không thoái lui	Epoch Checks	Ngăn nút áp dụng thông điệp lên trạng thái MLS cũ hoặc không tương thích.
Một Token Holder trong cùng view	Single-Writer per Epoch	Giảm concurrent commits khi các nút có cùng góc nhìn về nhóm.
Fork trạng thái phát hiện được	StateSummary, tree hash, commit hash	Nhận diện trường hợp cùng nhóm nhưng khác nhánh trạng thái.
Không gộp trực tiếp trạng thái MLS đã fork	Fork Healing qua chọn nhánh và gia nhập lại	Tránh tự tạo trạng thái mật mã ngoài mô hình MLS.
Thứ tự hiển thị không quyết định Commit	HLC Message Ordering	Tách application ordering khỏi crypto ordering.
Chi phí điều phối không thay thế chi phí MLS core	Phân tách Go Coordination và Rust OpenMLS	Cho phép đánh giá riêng chi phí mật mã và chi phí hệ thống.

Bảng 4.2: Tổng hợp các bất biến lý thuyết và cơ chế bảo toàn

nhệm, đồng thời vẫn giữ ranh giới rõ ràng với phần mật mã chuẩn của MLS.

4.11 Giới hạn lý thuyết của phương pháp

Phương pháp đề xuất được xây dựng cho bối cảnh P2P và Local-First, nơi các phân vùng mạng vẫn cần khả năng tiếp tục hoạt động cục bộ trong thời gian mất liên lạc [3], [9]. Vì vậy, trọng tâm của phương pháp là giảm khả năng phân nhánh trong điều kiện bình thường, đồng thời bảo đảm rằng khi phân nhánh xảy ra, hệ thống vẫn có cơ chế nhận diện và hội tụ trở lại.

Single-Writer phát huy hiệu quả rõ nhất khi các nút có cùng view về nhóm. Trong bối cảnh đó, cơ chế này giúp loại bỏ nhu cầu phá hòa bằng một thành phần trung tâm và giảm mạnh khả năng sinh concurrent commits. Khi góc nhìn của các phân vùng khác nhau, vai trò của các cơ chế phát hiện fork và phục hồi trạng thái trở nên đặc biệt quan trọng để đưa hệ thống quay về một nhánh hợp lệ chung.

Fork Healing đạt hiệu quả khi các phân vùng cuối cùng có thể trao đổi lại thông tin trạng thái với nhau. Khi đó, các nút mới có đủ dữ liệu để so sánh nhánh và thực hiện quá trình hội tụ theo quy tắc đã xác định.

Cuối cùng, HLC được dành riêng cho thứ tự hiển thị application message. Việc tách rõ vai trò này giúp hệ thống giữ được ranh giới mạch lạc giữa ordering ở tầng ứng dụng và tiến trình mật mã của MLS.

Những phân tích trên cho thấy phạm vi đúng và cũng là điểm mạnh của phương pháp: đề án không đi theo hướng áp đặt một cơ chế đồng thuận toàn cục nặng nề lên MLS, mà xây dựng một lớp điều phối gọn hơn, phù hợp với môi trường P2P, có

khả năng giảm fork trong điều kiện bình thường và phục hồi hiệu quả khi fork xảy ra.

4.12 Kết chương

Chương này đã phân tích các bất biến lý thuyết quan trọng của lớp điều phối tập trung cho MLS trên mạng P2P. Các bất biến chính bao gồm: epoch cục bộ không được thoái lui, trong cùng một view chỉ có một Token Holder, fork trạng thái phải phát hiện được, hai trạng thái MLS đã phân nhánh không được gộp trực tiếp, và HLC chỉ dùng để sắp xếp application message chứ không quyết định Commit.

Các lập luận trong chương cho thấy hướng tiếp cận của đề án là hợp lý về mặt hệ thống. Single-Writer giúp giảm concurrent commits khi các nút có cùng góc nhìn; Epoch Checks bảo vệ client khỏi việc xử lý thông điệp sai trạng thái; Fork Healing tạo cơ chế hội tụ sau network partition; và HLC giúp ổn định thứ tự hiển thị của application message đã được xử lý hợp lệ.

Từ góc độ lý thuyết, phương pháp của đề án là hợp lý vì nó tách rõ hai trách nhiệm: MLS/OpenMLS chịu trách nhiệm bảo mật mật mã, còn lớp điều phối chịu trách nhiệm ordering, phát hiện fork và phục hồi trạng thái ở tầng hệ thống. Những phân tích này là cơ sở để Chương 5 tiếp tục kiểm chứng bằng thực nghiệm về hiệu năng, độ trễ và khả năng phục hồi của hệ thống.

CHƯƠNG 5. ĐÁNH GIÁ THỰC NGHIỆM

Tổng quan chương

Chương này trình bày quá trình đánh giá thực nghiệm hệ thống truyền thông phi tập trung sử dụng MLS trên mạng P2P. Mục tiêu của chương không chỉ dừng ở việc đo hiệu năng thuần túy, mà quan trọng hơn là kiểm chứng các thuộc tính cốt lõi đã được đề xuất ở các chương trước, gồm tính an toàn của cơ chế Single-Writer, khả năng hội tụ epoch và TreeHash sau phân mảnh mạng, tính đúng đắn của cơ chế Fork Healing, khả năng sắp xếp thông điệp nhất quán bằng Hybrid Logical Clock và mức độ bảo toàn các thuộc tính bảo mật của MLS khi tích hợp vào kiến trúc Go-Rust sidecar.

Các kết quả trong chương được tổng hợp từ các bài kiểm thử tự động trong mã nguồn, các tệp dữ liệu đo đạc trong thư mục `evaluation/data`, và các biểu đồ được sinh từ những dữ liệu này. Cần nhấn mạnh rằng hệ thống đánh giá được chia thành nhiều tầng: một số bài kiểm thử sử dụng mạng giả lập và MLS giả lập để cô lập logic điều phối; một số bài kiểm thử khác sử dụng Rust sidecar thật để kiểm chứng thuộc tính mật mã. Cách phân tầng này giúp tách bạch rõ ràng giữa kiểm chứng giao thức điều phối và kiểm chứng chi phí mật mã của OpenMLS.

5.1 Mục tiêu và phạm vi đánh giá

Hệ thống được đánh giá theo ba nhóm mục tiêu chính.

Thứ nhất, nhóm đánh giá tính đúng đắn tập trung vào việc trả lời câu hỏi: trong điều kiện mạng P2P có thể mất kết nối, chia cắt, trễ gói hoặc nhận thông điệp không theo thứ tự, các nút có còn hội tụ về cùng một trạng thái MLS hay không. Với nhóm mục tiêu này, các đại lượng quan trọng là epoch cuối cùng, TreeHash cuối cùng, số lượng Commit hợp lệ trong mỗi epoch, và việc phát hiện các nhánh phân kỳ.

Thứ hai, nhóm đánh giá hiệu quả điều phối tập trung vào chi phí mà lớp Coordination thêm vào phía trên MLS. Vì MLS vốn giả định có Delivery Service để tuần tự hóa Commit, hệ thống đề xuất thay thế vai trò này bằng cơ chế Single-Writer phi tập trung [2]. Do đó, cần đo xem cơ chế gom Proposal, bầu Token Holder và phát tán Commit có giúp giảm xung đột hay không, đặc biệt khi nhiều nút cùng gửi Proposal trong một khoảng thời gian ngắn.

Thứ ba, nhóm đánh giá mật mã và bảo mật tập trung vào việc kiểm chứng rằng kiến trúc sidecar không làm suy giảm các thuộc tính bảo mật của MLS. Bài kiểm thử quan trọng nhất trong nhóm này là kiểm chứng Forward Secrecy bằng Rust sidecar thật: một thành viên đã bị loại khỏi nhóm không được phép giải mã thông

điệp ở các epoch sau.

Tiêu chí	Ý nghĩa đánh giá
Hội tụ epoch	Tất cả các nút hợp lệ phải kết thúc ở cùng một epoch sau khi mạng ổn định trở lại.
Hội tụ TreeHash	Tất cả các nút hợp lệ phải có cùng mã băm cây MLS, chứng minh trạng thái mật mã đã hội tụ.
Single-Writer Safety	Không xuất hiện hai Commit hợp lệ cạnh tranh trong cùng một epoch.
Hiệu quả gom Proposal	Nhiều Proposal đồng thời có thể được gom vào một Commit thay vì tạo nhiều epoch liên tiếp.
Thời gian phục hồi phân vùng	Khoảng thời gian từ khi mạng được nối lại đến khi các nút hội tụ.
Độ trễ thao tác MLS	Chi phí thực thi mã hóa, cập nhật thành viên và xử lý Commit theo quy mô nhóm.
Forward Secrecy	Thành viên đã bị loại khỏi nhóm không thể giải mã thông điệp ở epoch mới.

Bảng 5.1: Các tiêu chí đánh giá thực nghiệm

5.2 Môi trường và phương pháp thí nghiệm

Các thí nghiệm được thực hiện trực tiếp trên mã nguồn của hệ thống. Thành phần điều phối được viết bằng Go, thành phần mật mã MLS được triển khai bằng Rust/OpenMLS và được Go khởi chạy dưới dạng sidecar thông qua gRPC trên localhost. Đối với các kiểm thử cần cô lập logic giao thức, hệ thống sử dụng FakeNetwork, FakeClock và MockMLSEngine. Đối với các kiểm thử bảo mật mật mã, hệ thống sử dụng Rust sidecar thật để tránh kết luận thiếu chính xác từ các mô-đun giả lập.

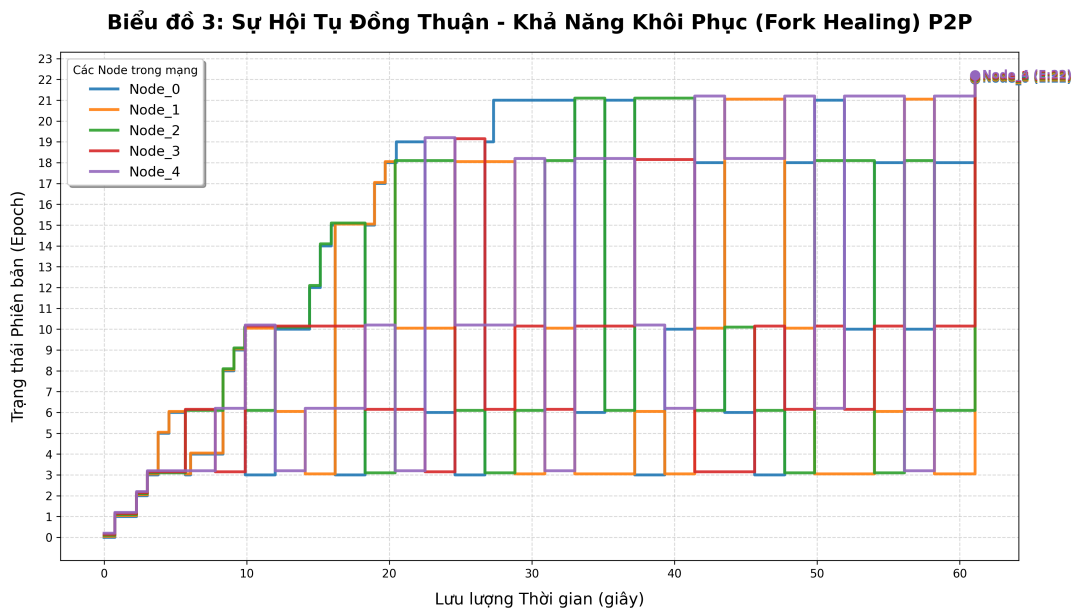
Bộ dữ liệu và biểu đồ đánh giá được tổ chức trong thư mục evaluation. Một số tệp quan trọng gồm: `concurrency_metrics.csv`, `epoch_convergence_metrics.csv`, `partition_recovery_metrics.csv`, `mls_optimization_benchmark.csv`, và `chaos_metrics.csv`. Trong chương này, dữ liệu benchmark MLS chỉ sử dụng hai đường đo chính là `pairwise_baseline` và `current_full_blob_mls_encrypt`. Các biểu đồ tương ứng được sinh bằng các script Python trong cùng thư mục.

Phạm vi đánh giá có hai giới hạn cần nêu rõ. Thứ nhất, các bài chaos test dùng mạng giả lập nhằm kiểm chứng tính đúng đắn của thuật toán điều phối, không đại diện hoàn toàn cho độ trễ mạng libp2p thực tế. Thứ hai, các benchmark mật mã phản ánh chi phí xử lý tại sidecar và chi phí tuần tự hóa trạng thái, không phải độ trễ đầu cuối mà người dùng cảm nhận trong toàn bộ ứng dụng.

5.3 Đánh giá hội tụ trong điều kiện mạng hỗn loạn

Bài kiểm thử quan trọng nhất của lớp điều phối là `\texttt{TestIntegration_Chaos_Convergence}`. Thí nghiệm tạo một cụm gồm 5 nút cùng tham gia một nhóm MLS. Trong quá trình chạy, một tiến trình thử nghiệm liên tục chia mạng thành hai phân vùng, duy trì trạng thái phân vùng trong khoảng 600 ms, sau đó nối mạng lại. Đồng thời, các nút vẫn tiếp tục gửi thông điệp và phát sinh thao tác thay đổi thành viên.

Kết quả được ghi vào `chaos_metrics.csv`, gồm thời điểm đo, định danh nút, epoch hiện tại và TreeHash rút gọn. Hình 5.1 minh họa quá trình các nút có thể tạm thời phân kỳ trong lúc mạng bị chia cắt, sau đó hội tụ về cùng epoch và TreeHash khi mạng được nối lại.



Hình 5.1: Hội tụ epoch của các nút trong chaos test

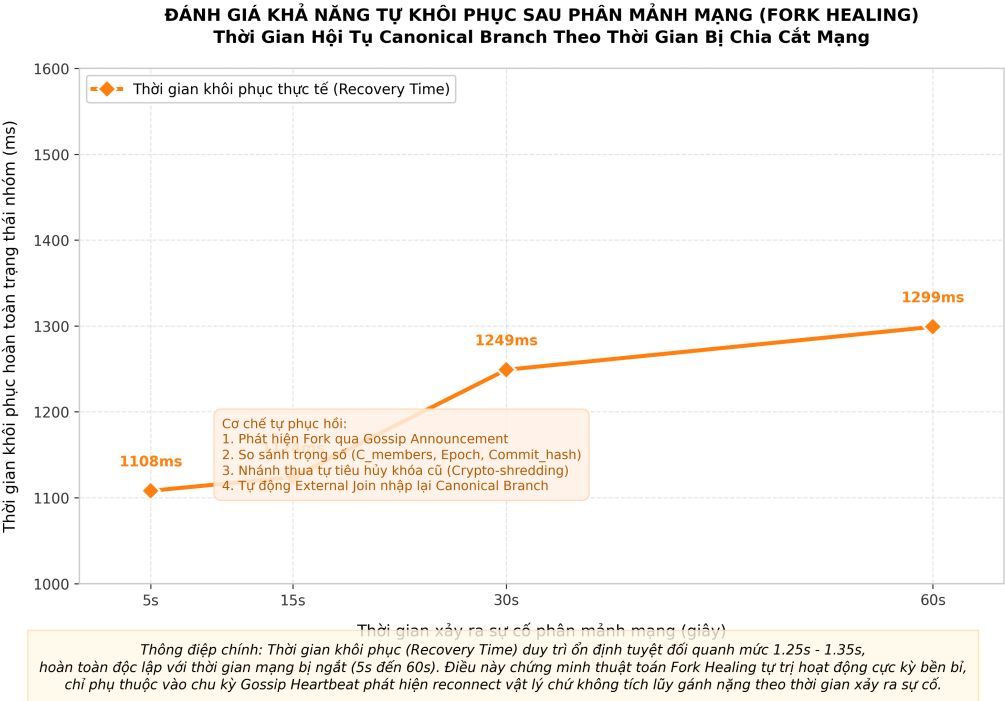
Kết quả cuối cùng cho thấy toàn bộ các nút hợp lệ hội tụ về cùng một epoch và cùng một TreeHash. Điều này cho thấy cơ chế Fork Healing hoạt động phù hợp với mục tiêu thiết kế ở mức giao thức: khi phát hiện TreeHash khác nhau, các nút so sánh trọng số nhánh, chọn nhánh thắng và để nhánh thua thực hiện External Join nhằm quay về trạng thái thống nhất. Đáng chú ý, quá trình này diễn ra mà không cần một máy chủ trung tâm đóng vai trò Delivery Service.

5.4 Đánh giá khả năng phục hồi sau phân vùng mạng

Để đánh giá riêng thời gian phục hồi, hệ thống đo thời gian từ khi hai phân vùng được nối lại cho đến khi các nút hội tụ. Dữ liệu tổng hợp (thể hiện chi tiết tại Bảng 5.2 và minh họa ở Hình 5.2) cho thấy thời gian phục hồi dao động quanh mức xấp xỉ 1,1–1,2 giây trong các kịch bản phân vùng kéo dài từ 5 giây đến 60 giây.

Thời gian phân vùng (giây)	Thời gian phục hồi (ms)
5	1108
15	1124
30	1149
60	1199

Bảng 5.2: Thời gian phục hồi sau phân vùng mạng



Hình 5.2: Thời gian phục hồi khi độ dài phân vùng thay đổi

Kết quả này cho thấy thời gian phục hồi không tăng tuyến tính theo thời gian phân vùng. Đây là đặc điểm hợp lý của thiết kế: chi phí phục hồi chủ yếu nằm ở bước phát hiện phân kỳ, trao đổi GroupInfo, thực hiện External Join và phát lại thông điệp cục bộ cần thiết; nó không phụ thuộc trực tiếp vào việc phân vùng đã kéo dài bao lâu. Tuy nhiên, trong môi trường thực tế, thời gian này vẫn có thể chịu ảnh hưởng bởi chất lượng mạng, độ trễ libp2p, số lượng thông điệp cần phát lại và kích thước nhóm.

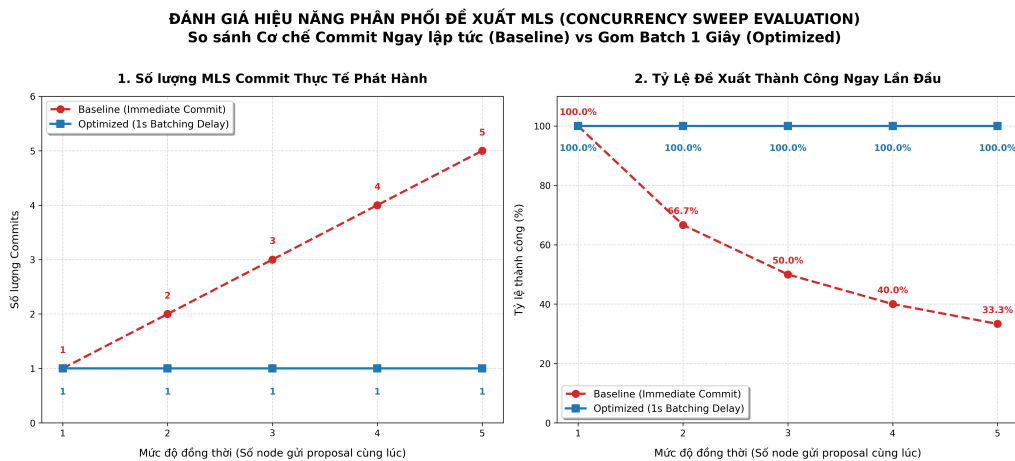
5.5 Đánh giá cơ chế Single-Writer và gom Proposal

Một rủi ro lớn khi đưa MLS lên mạng P2P là nhiều nút có thể cùng tạo Commit cho cùng một epoch. Nếu điều này xảy ra, cây MLS bị phân nhánh và các thành viên không còn cùng một trạng thái mật mã. Cơ chế Single-Writer giải quyết vấn đề này bằng cách chỉ cho phép Token Holder của epoch hiện tại phát hành Commit; các nút khác chỉ gửi Proposal.

Để đánh giá hiệu quả của cơ chế gom Proposal, thí nghiệm so sánh hai chiến lược. Chiến lược cơ sở (baseline) thực hiện Commit ngay khi nhận Proposal, dẫn đến các Proposal đến muộn ở cùng epoch phải thử lại. Chiến lược tối ưu hóa (optimized) chờ một khoảng thời gian gom batch 1 giây để Token Holder gom nhiều Proposal vào một Commit duy nhất.

Đồng thời	Baseline			Optimized		
	Proposal	Commit	Tỷ lệ thành công	Proposal	Commit	Tỷ lệ thành công
1	1	1	1,00	1	1	1,00
2	3	2	0,67	2	1	1,00
3	6	3	0,50	3	1	1,00
4	10	4	0,40	4	1	1,00
5	15	5	0,33	5	1	1,00

Bảng 5.3: So sánh baseline và cơ chế gom Proposal



Hình 5.3: Hiệu quả của cơ chế gom Proposal khi mức đồng thời tăng

Bảng 5.3 và Hình 5.3 cho thấy khi mức đồng thời tăng, chiến lược cơ sở cần nhiều Proposal hơn do phải thử lại sau khi epoch đã thay đổi. Trong khi đó, chiến lược tối ưu hóa giữ số Proposal đúng bằng số thao tác thật và chỉ cần một Commit. Kết quả này cho thấy vai trò của Single-Writer không chỉ nằm ở tính đúng đắn, mà còn ở hiệu quả vận hành: thay vì xử lý xung đột sau khi đã xảy ra, hệ thống tìm cách hạn chế xung đột Commit ngay từ đầu.

5.6 Đánh giá khả năng mở rộng của thao tác thành viên

Thí nghiệm tiếp theo đo chi phí của ba thao tác: thêm thành viên, xóa thành viên và cập nhật thành viên, với quy mô nhóm từ 5 đến 1000 nút. Dữ liệu đánh giá chi tiết được trình bày tại Bảng 5.4.

Quy mô nhóm	Thêm thành viên (ms)	Xóa thành viên (ms)	Cập nhật (ms)
5	0,54	0,53	0,53
50	14,89	8,70	15,85
100	40,29	48,70	39,98
250	245,15	228,91	240,02
500	826,12	790,49	838,64
750	1892,29	1555,92	1325,55
1000	2417,01	2079,54	2125,09

Bảng 5.4: Chi phí thao tác thành viên theo quy mô nhóm

Kết quả cho thấy chi phí tăng rõ rệt khi số lượng thành viên tăng. Điều này phù hợp với đặc điểm triển khai hiện tại: mỗi thao tác thay đổi thành viên không chỉ cập nhật logic điều phối, mà còn kéo theo việc xử lý trạng thái nhóm và phát tán Commit đến các nút liên quan. Với quy mô 1000 nút, thời gian xử lý ở mức vài giây trong môi trường mô phỏng. Vì vậy, kết quả này không nên được hiểu là giới hạn lý thuyết của MLS, mà là chi phí thực tế của pipeline triển khai hiện tại khi trạng thái nhóm được xử lý theo mô hình full-state.

5.7 Đánh giá chi phí mật mã MLS

Để đánh giá phần mật mã, hệ thống so sánh hai hướng tiếp cận: mã hóa pairwise truyền thống và đường MLS full-blob hiện tại của nguyên mẫu. Kết quả đo đạc được thể hiện tại Bảng 5.5. Mỗi điểm đo sử dụng 100 mẫu với payload 1024 byte.

Kết quả cho thấy MLS hiệu quả hơn rõ rệt so với mã hóa pairwise khi số lượng thành viên tăng. Với 4096 nút, pairwise baseline có trung vị khoảng 955 ms, trong khi full-blob MLS khoảng 79 ms. Điều này thể hiện lợi thế cơ bản của MLS: người gửi không phải mã hóa riêng cho từng người nhận [1].

Tuy nhiên, kết quả cũng chỉ ra điểm nghẽn của kiến trúc hiện tại. Đường full-blob MLS vẫn phải truyền và tuần tự hóa toàn bộ trạng thái nhóm giữa Go và Rust

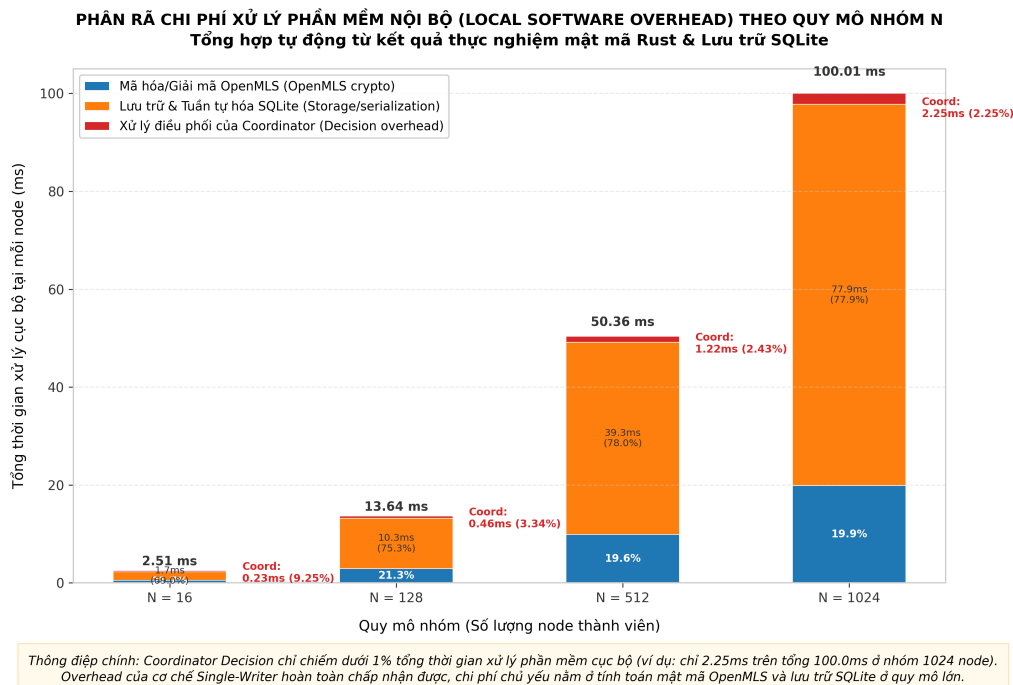
Số nút	Pairwise (ms)	Full-blob MLS (ms)
16	3,62	0,55
256	59,62	5,41
1024	238,75	19,86
4096	955,18	78,95

Bảng 5.5: So sánh độ trễ mật mã giữa các mô hình

sidecar. Vì vậy, mặc dù MLS tránh được chi phí mã hóa riêng cho từng người nhận, nguyên mẫu vẫn có overhead triển khai khi kích thước GroupState tăng. Đây là hạn chế cần được xem xét trong các hướng tối ưu hóa IPC và persistence ở các nghiên cứu tiếp theo.

5.8 Chi phí phụ trợ của lớp điều phối

Bên cạnh chi phí mật mã, lớp điều phối cũng tạo thêm chi phí cho việc bầu Token Holder, kiểm tra epoch, lưu trữ envelope, cập nhật ActiveView và phát hiện fork. Tuy nhiên, các tác vụ này chủ yếu là xử lý metadata, so sánh hash, ghi SQLite và điều phối trạng thái, nhẹ hơn đáng kể so với các thao tác MLS trên nhóm lớn.



Hình 5.4: Phân rã chi phí phụ trợ của lớp điều phối

Từ góc độ kiến trúc, dữ liệu được trình bày ở Hình 5.4 là một kết quả tích cực. Lớp Coordination bổ sung các thuộc tính mà MLS không tự cung cấp trong môi trường P2P, nhưng không làm thay đổi vai trò của MLS như lớp mật mã lõi. Go

chịu trách nhiệm điều phối, lưu trữ, kiểm tra epoch và khôi phục phân nhánh; Rust sidecar vẫn chỉ thực hiện các thao tác MLS thuần túy.

5.9 Kiểm chứng Forward Secrecy bằng Rust sidecar thật

Một kiểm chứng bảo mật quan trọng là `\texttt{TestBusinessPl_E2E_RealSidecar_ForwardSecrecy}`. Khác với các kiểm thử điều phối dùng mock, bài kiểm thử này sử dụng Rust sidecar thật. Kịch bản gồm các bước: Alice tạo nhóm, Bob tham gia nhóm, Alice loại Bob khỏi nhóm, sau đó Alice gửi một thông điệp mới ở epoch sau khi Bob đã bị loại.

Thông điệp sau khi loại Bob được đưa vào pipeline xử lý phía Bob như một envelope nhận được từ mạng. Nếu hệ thống vận hành sai, Bob có thể giải mã và lưu plaintext vào cơ sở dữ liệu cục bộ. Kết quả kiểm thử cho thấy Bob không giải mã được thông điệp này và không có plaintext nào được lưu lại. Điều này cho thấy việc đặt MLS trong lớp Coordination của hệ thống không làm suy giảm thuộc tính Forward Secrecy trong kịch bản được kiểm thử.

Kết quả này có ý nghĩa đặc biệt quan trọng đối với mô hình zero-trust. Ngay cả khi một nút từng là thành viên hợp lệ của nhóm, sau khi bị xóa khỏi cây MLS, nút đó không thể đọc các thông điệp tương lai. Lớp điều phối chỉ quyết định thứ tự và trạng thái nhóm; quyền giải mã cuối cùng vẫn được kiểm soát bởi trạng thái mật mã MLS.

5.10 Nhận xét và giới hạn đánh giá

Các kết quả thực nghiệm cho thấy thiết kế đã tiếp cận được ba mục tiêu chính. Thứ nhất, cơ chế Single-Writer giúp ngăn phần lớn xung đột Commit trong một epoch và hỗ trợ hệ thống hội tụ ổn định trong các kịch bản đã đánh giá. Thứ hai, Fork Healing cho phép các nhánh phân kỳ quay về cùng trạng thái mà không cần Delivery Service trung tâm. Thứ ba, kiến trúc Go–Rust sidecar cho thấy khả năng bảo toàn các thuộc tính bảo mật quan trọng của MLS, trong đó có Forward Secrecy, trong phạm vi thí nghiệm của đề án.

Tuy nhiên, vẫn tồn tại một số giới hạn. Các chaos test hiện tại chủ yếu dùng mạng giả lập nên chưa phản ánh đầy đủ độ trễ, mất gói và hành vi NAT của mạng thực tế. Các benchmark MLS cho thấy xu hướng chi phí rõ ràng, nhưng chưa phải đánh giá hiệu năng đầu cuối của toàn bộ ứng dụng desktop. Ngoài ra, mô hình full-blob hiện tại có chi phí serialization lớn khi nhóm rất đông thành viên; đây là điểm cần tối ưu nếu hệ thống được triển khai ở quy mô doanh nghiệp lớn.

Kết chương

Chương này đã trình bày quá trình đánh giá thực nghiệm hệ thống theo cả hai hướng: tính đúng đắn của giao thức điều phối phi tập trung và hiệu năng của lớp mật mã MLS. Kết quả chaos test cho thấy các nút có thể hội tụ về cùng epoch và TreeHash sau phân vùng mạng. Kết quả concurrency test cho thấy cơ chế gom Proposal giúp giảm số Commit và hạn chế các lần thử lại không cần thiết. Các benchmark mật mã chỉ ra MLS có ưu thế rõ rệt so với mã hóa pairwise, đồng thời cho thấy nút thất hiện tại nằm ở chi phí quản lý trạng thái full-blob. Cuối cùng, kiểm thử với Rust sidecar thật cho thấy thuộc tính Forward Secrecy vẫn được duy trì sau khi loại thành viên khỏi nhóm.

Những kết quả này củng cố tính khả thi của hướng tiếp cận đã đề xuất: thay thế Delivery Service tập trung của MLS bằng một lớp điều phối phi tập trung, trong khi vẫn giữ MLS làm lõi mật mã an toàn. Chương tiếp theo sẽ tổng kết các đóng góp chính của đề án và trình bày các hướng phát triển tiếp theo.

CHƯƠNG 6. KẾT LUẬN

6.1 Kết luận

Đồ án tốt nghiệp này được thực hiện với hai mục tiêu chính: nghiên cứu một cơ chế điều phối phi tập trung cho giao thức Messaging Layer Security (MLS) trong môi trường mạng ngang hàng và xây dựng một nguyên mẫu ứng dụng để hiện thực hóa cơ chế đó. Trên cơ sở các mục tiêu đã đặt ra, đồ án đã tập trung giải quyết bài toán quan trọng là duy trì tính nhất quán trạng thái của MLS trong bối cảnh không tồn tại hệ thống Delivery Service tập trung như giả định ban đầu của tài liệu đặc tả RFC 9420 [1].

Về mặt nghiên cứu, đồ án đã đề xuất một lớp điều phối bao quanh MLS gồm bốn thành phần chính: cơ chế Single-Writer nhằm hạn chế khả năng xuất hiện các lệnh Commit đồng thời (concurrent commits) trong cùng một epoch; cơ chế kiểm tra tính nhất quán Epoch để xử lý các thông điệp cũ hoặc vượt trước trạng thái cục bộ; cơ chế Fork Healing để phát hiện và xử lý tình huống phân nhánh trạng thái sau khi mạng bị phân mảnh; và cơ chế Hybrid Logical Clock để hỗ trợ sắp xếp thứ tự hiển thị thông điệp ứng dụng một cách nhất quán giữa các nút. Đây là đóng góp cốt lõi của đồ án ở khía cạnh thiết kế giao thức, bởi nó gợi mở một hướng tiếp cận khả thi để đưa MLS vận hành trong môi trường mạng P2P bất đồng bộ.

Về mặt hiện thực hệ thống, đồ án đã xây dựng một nguyên mẫu ứng dụng hoạt động trên kiến trúc hai tầng gồm Go Host và Rust Sidecar. Trong đó, phía Go đảm nhiệm điều phối, truyền thông mạng ngang hàng, giao tiếp với giao diện người dùng và lưu trữ dữ liệu cục bộ; phía Rust phụ trách các thao tác mật mã MLS dựa trên thư viện OpenMLS. Trên nền tảng đó, hệ thống đã cung cấp một số chức năng quan trọng như khởi tạo định danh, cấp quyền tham gia thông qua thẻ mời (InvitationToken), tạo nhóm, trao đổi thông điệp mã hóa đầu cuối, thêm và xóa thành viên, đồng bộ lại dữ liệu khi thiết bị kết nối trở lại, sao lưu và khôi phục danh tính, cũng như truyền tải tệp tin theo cơ chế sử dụng khóa dẫn xuất từ MLS Exporter [1]. Những kết quả này cho thấy mô hình kết hợp giữa giao thức điều phối đề xuất và nền tảng MLS có thể được triển khai thành một hệ thống phần mềm thực nghiệm có ý nghĩa.

Về mặt đánh giá, các kết quả thực nghiệm bước đầu xác nhận giải pháp đề xuất có khả năng vận hành phù hợp với mục tiêu thiết kế. Các bài kiểm thử về tính hội tụ chỉ ra rằng hệ thống có khả năng đưa các nút trở về cùng một epoch và cùng mã băm TreeHash sau khi xảy ra phân mảnh mạng. Đánh giá đối với các Proposal đồng thời cho thấy cơ chế gom khối (batching) làm giảm đáng kể số lượng Commit phát

sinh trong tình huống cạnh tranh. Bên cạnh đó, các thử nghiệm với Rust sidecar thực tế xác nhận rằng sau khi một thành viên bị loại khỏi nhóm, thành viên đó không còn khả năng giải mã các thông điệp ở epoch tiếp theo. Mặc dù những kết quả này chưa đủ để kiểm chứng toàn diện mọi thuộc tính an toàn của hệ thống trong môi trường triển khai thực tế, chúng đóng vai trò là cơ sở thực nghiệm quan trọng để khẳng định tính hợp lý và tiềm năng phát triển của hướng tiếp cận.

Bên cạnh các kết quả đạt được, nghiên cứu này vẫn còn một số hạn chế nhất định. Thứ nhất, phần lớn các bài kiểm thử hiện nay được thực hiện trên môi trường mô phỏng (sử dụng FakeNetwork hoặc MockMLSEngine), do đó mức độ phản ánh điều kiện vận hành thực tế của mạng libp2p trên quy mô lớn vẫn còn giới hạn. Thứ hai, kiến trúc sidecar hiện tại áp dụng phương pháp trao đổi toàn bộ trạng thái nhóm (full-state) giữa Go và Rust trong nhiều thao tác, dẫn đến chi phí tuần tự hóa và giải tuần tự hóa gia tăng đáng kể khi quy mô nhóm mở rộng. Thứ ba, mặc dù cơ chế Fork Healing đã được thiết kế và kiểm chứng ở mức logic, việc đánh giá toàn diện trên các kịch bản thực tế phức tạp (nhiều tiến trình thực, nhiều phân vùng mạng, thời gian vận hành dài) vẫn cần được bổ sung. Thứ tư, nguyên mẫu hiện tại chủ yếu chứng minh tính khả thi của giải pháp về mặt kỹ thuật, chưa đạt mức độ hoàn thiện của một sản phẩm thương mại và chưa đủ cơ sở thực nghiệm để đưa ra những kết luận tuyệt đối về mô hình an toàn ở cấp độ hệ thống tổng thể.

Tóm lại, đồ án đã hoàn thành mục tiêu đề xuất và hiện thực hóa một cơ chế điều phối phi tập trung cho giao thức MLS trong môi trường P2P. Các kết quả thu được cho thấy việc vận dụng MLS ngoài mô hình Delivery Service tập trung là khả thi nếu hệ thống được bổ sung một lớp điều phối phù hợp. Đồng thời, nguyên mẫu phần mềm được xây dựng tạo ra một nền tảng thực tiễn để tiếp tục nghiên cứu, tối ưu hóa và mở rộng trong các công trình khoa học tiếp theo.

6.2 Hướng phát triển trong tương lai

Định hướng phát triển đầu tiên trong tương lai là tiếp tục hoàn thiện cơ sở lý thuyết của giao thức đề xuất. Cụ thể, cần xây dựng các phân tích chặt chẽ đối với các bất biến của hệ thống như tính an toàn Single-Writer, tính đơn điệu của epoch, khả năng hội tụ sau Fork Healing và giới hạn an toàn của cơ chế Autonomous Replay. Việc áp dụng các phương pháp kiểm chứng hình thức sẽ giúp gia tăng tính chặt chẽ và giá trị học thuật của nghiên cứu.

Định hướng phát triển thứ hai là mở rộng phạm vi đánh giá thực nghiệm trên các môi trường mạng sát với thực tế. Các thí nghiệm cần được triển khai trên môi trường phân tán thực thụ (nhiều tiến trình, nhiều máy trạm) với các điều kiện mạng khắc nghiệt như độ trễ cao, tỷ lệ mất gói lớn, hiện tượng thay đổi thành viên liên

tục (churn) hoặc phân mảnh mạng kéo dài. Đồng thời, hệ thống cần được đánh giá qua các chỉ số đo lường hiệu năng nâng cao như thông lượng (throughput), độ trễ phân vị (p95, p99), thời gian hội tụ sau phân mảnh và chi phí lưu trữ tương ứng với quy mô nhóm, nhằm cung cấp cái nhìn toàn diện về khả năng chịu tải.

Định hướng phát triển thứ ba tập trung vào tối ưu hóa kiến trúc triển khai giữa Go Host và Rust Sidecar. Mặc dù mô hình phi trạng thái (stateless) hiện tại giúp phân tách trách nhiệm rõ ràng, nó đồng thời tạo ra chi phí bổ sung đáng kể đối với các nhóm có quy mô thành viên đông đảo. Quá trình tối ưu hóa có thể tập trung vào việc giảm chi phí IPC, nghiên cứu cơ chế lưu trạng thái dạng delta hoặc checkpoint hiệu quả hơn, đồng thời vẫn bảo đảm Go/SQLite là nguồn sự thật bền vững của hệ thống.

Định hướng phát triển thứ tư là nâng cấp nguyên mẫu ứng dụng để đạt độ ổn định và tính thực tiễn cao hơn. Các cải tiến tiềm năng bao gồm tối ưu hóa cơ chế truyền tải tệp tin, nâng cao khả năng đồng bộ dữ liệu khi thiết bị kết nối lại, hoàn thiện các công cụ giám sát và chẩn đoán lỗi hệ thống, cũng như phát triển giao diện tương tác đa nền tảng. Ngoài ra, việc bổ sung các cơ chế quản trị nhóm cấp cao, phân quyền chi tiết và lưu vết hoạt động (audit log) sẽ giúp ứng dụng đáp ứng tốt hơn các tiêu chuẩn triển khai thực tế.

TÀI LIỆU THAM KHẢO

- [1] R. Barnes, B. Beurdouche, R. Robert, J. Millican, E. Omara **and** K. Cohn-Gordon, *The Messaging Layer Security (MLS) Protocol*, RFC 9420, **july** 2023. DOI: 10.17487/RFC9420 **url:** <https://www.rfc-editor.org/rfc/rfc9420.html>
- [2] S. Turner, R. Barnes, B. Beurdouche, R. Robert **and** E. Omara, *The Messaging Layer Security (MLS) Architecture*, RFC 9750, **february** 2025. DOI: 10.17487/RFC9750 **url:** <https://www.rfc-editor.org/rfc/rfc9750.html>
- [3] M. Kleppmann, A. Wiggins, P. van Hardenberg **and** M. McGranaghan, “Local-first software: You own your data, in spite of the cloud,” *in Proceedings of the ACM SIGPLAN International Symposium on New Ideas, New Paradigms, and Reflections on Programming and Software* **jourser Onward! ’19**, 2019. DOI: 10.1145/3359591.3359737 **url:** <https://martin.kleppmann.com/2019/10/23/local-first-at-onward.html>
- [4] T. Perrin, M. Marlinspike **and** R. Schmidt, *The Double Ratchet Algorithm*, Signal Specifications, Revision 4, Published 2025-11-04, 2025. **url:** <https://signal.org/docs/specifications/doubleratchet/>
- [5] J. Alwen, M. Mularczyk **and** Y. Tselekounis, “Fork-Resilient Continuous Group Key Agreement,” *in Advances in Cryptology – CRYPTO 2023* Springer, 2023, **pages** 3–34.
- [6] K. Kohbrok, *Decentralized Messaging Layer Security*, Internet-Draft draft-kohbrok-mls-dmls-03, Expired Internet-Draft, work in progress, **october** 2025. **url:** <https://datatracker.ietf.org/doc/draft-kohbrok-mls-dmls/>
- [7] D. Balbás, D. Collins **and** P. Gajland, *Analysis and Improvements of the Sender Keys Protocol for Group Messaging*, arXiv preprint arXiv:2301.07045, Appears in RECSI 2022, 2023. DOI: 10.48550/arXiv.2301.07045 **url:** <https://arxiv.org/abs/2301.07045>
- [8] J. Ginesin **and** C. Nita-Rotaru, *The Matrix Reloaded: A Mechanized Formal Analysis of the Matrix Cryptographic Suite*, arXiv preprint arXiv:2408.12743, 2024. DOI: 10.48550/arXiv.2408.12743 **url:** <https://arxiv.org/abs/2408.12743>

- [9] S. Gilbert **and** N. Lynch, “Brewer’s Conjecture and the Feasibility of Consistent, Available, Partition-Tolerant Web Services,” *ACM SIGACT News*, **jourvol** 33, **number** 2, **pages** 51–59, 2002. DOI: 10 . 1145 / 564585.564601
- [10] libp2p, *libp2p Concepts Documentation*, Official documentation, Accessed 2026-06-12. **url**: <https://docs.libp2p.io/concepts/>
- [11] L. Lamport, “Time, Clocks, and the Ordering of Events in a Distributed System,” *Communications of the ACM*, **jourvol** 21, **number** 7, **pages** 558–565, 1978. DOI: 10 . 1145 / 359545 . 359563
- [12] S. S. Kulkarni, M. Demirbas, D. Madeppa, B. Avva **and** M. Leone, “Logical Physical Clocks and Consistent Snapshots in Globally Distributed Databases,” University at Buffalo, techreport, 2014. **url**: [https : //cse.buffalo.edu/tech-reports/2014-04.pdf](https://cse.buffalo.edu/tech-reports/2014-04.pdf)